SISTEMAS DE INGENIERÍA / GUÍAS DE USO

ArquitecturaRepo

Manual completo de uso

VERSIÓN 1.4 / ESPAÑOL

Ocho capítulos extensos: descubrimiento, instalación, adopción, operación, cambios, recuperación, aceptación y ejemplo guiado.

MATÍAS ACUÑA · NOVIEMBRE DE 2026

Documentación contrastada con las fuentes de ArquitecturaRepo.

Índice general

00 — Antes de empezar **5**
Contenido del capítulo . . . 5
1. Qué vas a instalar y qué no . . . 5
2. Quién toma cada decisión . . . 6
3. La información que debe permanecer accesible . . . 7
4. Cómo leer el manual completo . . . 8
5. Preparación: las decisiones que sí son tuyas . . . 9
6. Alcance, límites y versiones . . . 10
7. Un recorrido de extremo a extremo . . . 10
8. Decisiones habituales y sus responsables . . . 10
9. Ficha de arranque de una página . . . 11
10. Cómo evaluar si el sistema te está ayudando . . . 11
Fuentes y vigencia . . . 12

01 — Instalación y configuración **13**
Contenido del capítulo . . . 13
1. Entorno y cuentas . . . 13
2. Starter, fork y aplicación: tres ubicaciones distintas . . . 14
3. Verificar rutas antes de escribir . . . 14
4. Elegir un perfil proporcional . . . 15
5. Crear el control layer . . . 16
6. Validar sin inventar aceptación . . . 16
7. La próxima acción es discovery, no programar . . . 17
8. Antes de repetir una instalación . . . 17
9. Del scaffold a una adopción real . . . 17
10. Conexiones, permisos y privacidad . . . 18
11. Diagnóstico seguro . . . 18
Fuentes y vigencia . . . 19

02 — El Planner y la adopción del proyecto **20**
Contenido del capítulo . . . 20
1. Una conversación no es el proyecto . . . 20
2. Preparar el espacio de trabajo . . . 21
3. Discovery: hacer las preguntas correctas . . . 21
4. Baseline técnico y plan de entrega . . . 22
5. Adoption: conectar el starter con el proyecto real . . . 23
6. El examen decisivo: contexto fresco . . . 23
7. Primera campaña autorizada . . . 24
8. Ejemplo de sesión inicial con el Planner . . . 24
9. Un baseline que el CLIENT pueda confirmar . . . 24
10. Ejemplo de primera campaña . . . 25
Fuentes y vigencia . . . 26

03 — Operación diaria **27**
Contenido del capítulo . . . 27
1. Preparar una sesión sin empezar de nuevo . . . 27
2. Convertir un deseo en una capacidad comprobable . . . 28
3. Qué hace I3 cuando el trabajo se complica . . . 28

4. Autorizaciones y efectos externos . . . 29
5. Cómo leer críticamente un reporte del ejecutor . . . 29
6. Git y la continuidad . . . 30
7. Cierre diario en cinco preguntas . . . 30
8. Una jornada sin depender de la memoria del chat . . . 30
9. Ejemplo de despacho delimitado . . . 31
10. Interpretar estados de un controlador . . . 31
11. Procesar el reporte final . . . 32
12. Dejar una sesión recuperable . . . 32
Fuentes y vigencia . . . 32

04 — Solicitar cambios 33
Contenido del capítulo . . . 33
1. Empezar por el resultado, no por el código . . . 33
2. Una ficha breve para formular la necesidad . . . 33
3. Cuáles son tus preguntas y cuáles no . . . 34
4. Cambiar de opinión sin destruir el historial . . . 34
5. Pedir calidad sin imponer una tecnología . . . 35
6. Priorizar en vez de abrir diez Goals . . . 35
7. Cuando el Planner necesita una decisión real . . . 36
8. Cinco requisitos bien formulados . . . 36
9. Cambio de producto frente a defecto de implementación . . . 37
10. Qué decir cuando «funciona» pero no sirve . . . 38
11. Guion corto para comunicar una prioridad . . . 38
Fuentes y vigencia . . . 38

05 — Errores, incidentes y recuperación 39
Contenido del capítulo . . . 39
1. Primero conservar evidencia . . . 39
2. Cuando falla el entorno o una cuenta . . . 40
3. Si no valida el control layer . . . 40
4. Git no es un botón para «deshacer problemas» . . . 41
5. Un fallo repetido exige cambiar la causa . . . 41
6. Seguridad, privacidad y derechos de terceros . . . 42
7. Salir de un incidente con pruebas . . . 42
8. Tres incidentes completos y su recuperación . . . 42
9. Matriz para operaciones sensibles . . . 43
10. Qué no demuestra una validación verde . . . 44
11. Privacidad y feedback del starter . . . 44
Fuentes y vigencia . . . 45

06 — Cierre de capacidades y aceptación 46
Contenido del capítulo . . . 46
1. Una capacidad es un resultado de producto . . . 46
2. Los cinco componentes del cierre . . . 47
3. El candidato exacto como unidad de revisión . . . 48
4. Evidencia insuficiente sin campañas artificiales . . . 48
5. Aceptar, recuperar o cambiar el objetivo . . . 49
6. Revisiones periódicas y regla de detención . . . 49
7. Registro de cierre reconstruible . . . 50
8. Ejemplo de rechazo y corrección . . . 50
9. Producto utilizable frente a framework verde . . . 51
10. Qué hacer después del cierre . . . 51

Fuentes y vigencia 52

07 — Ejemplo guiado: una aplicación de gestión para un club 53
Contenido del capítulo 53
1. El producto que quiere construir el club 53
2. Descubrir preguntas que cambian el resultado 54
3. Instalar el control sin confundirlo con la app 54
4. Primera capacidad: gestión de socios 55
5. Del Planner a I3: una solicitud operable 56
6. Qué probar antes de aceptar 56
7. Segunda capacidad: cuotas y pagos 57
8. Evaluar el ejemplo completo 58
9. Primer ciclo con datos ficticios 58
10. Registrar y enrutar un defecto 58
11. Dependencias del segundo ciclo 59
12. Ejercicio final de recuperación 59
Fuentes y vigencia 60

00 — Antes de empezar

ArquitecturaRepo · Manual completo de uso · Edición v1.4 · Español

> Este capítulo presenta la finalidad, los límites y las responsabilidades del sistema. Corresponde a la versión pública v1.4.0. Su contenido es una guía de uso, no una prueba de que una aplicación particular haya sido construida, asegurada o aceptada. Los comandos vigentes se encuentran en la guía de primera instalación y las notas de release.

Contenido del capítulo

1. Qué vas a instalar y qué no
2. Quién toma cada decisión
3. La información que debe permanecer accesible
4. Cómo leer el manual completo
5. Preparación: las decisiones que sí son tuyas
6. Alcance, límites y versiones
7. Un recorrido de extremo a extremo
8. Decisiones habituales y sus responsables
9. Ficha de arranque de una página
10. Cómo evaluar si el sistema te está ayudando

1. Qué vas a instalar y qué no

ArquitecturaRepo es un starter público y portable de control de ingeniería. Reúne protocolos de operación, plantillas, perfiles de instalación, contratos comprobables, herramientas Python de validación y recuperación, y un controlador local de campañas opcional. Su función es mantener organizado un proyecto de software incluso cuando el trabajo pasa entre personas, agentes de programación, revisores y conversaciones nuevas.

No es una aplicación ya terminada ni un servicio al que le entregás una idea y recibís automáticamente un producto validado. Instalarlo no programa la aplicación, no crea un equipo de agentes competente por sí solo y no reemplaza las decisiones del dueño del producto. La aplicación que quieras construir necesita sus propios requisitos, elecciones técnicas, implementación real, pruebas, análisis de riesgos y aceptación.

La versión pública se identifica mediante un tag y artefactos de release concretos, no únicamente por el texto de un README. La v1.4.0 se publicó con un ZIP portable, checksum SHA-256 y manifest. Antes de instalar un archivo descargado, verificá su identidad. La rama `main` puede contener correcciones posteriores que no forman parte del ZIP ya publicado. Que el framework haya pasado sus pruebas significa que fueron comprobados ciertos mecanismos del framework: no demuestra que funcione una aplicación distinta construida con ellos.

El repositorio contiene ejemplos sintéticos y una aplicación de referencia para ejercitar partes del sistema. Esos ejemplos no son un Product State real, ni requisitos confirmados de una empresa, ni resultados que deban copiarse como si fueran tuyos. Cuando una capacidad de tu aplicación todavía no se verificó físicamente,

corresponde dejarla como desconocida, pendiente o bloqueada según el caso; no marcarla como terminada porque un agente lo anunció.

El beneficio buscado es continuidad. Una sesión nueva tendría que poder inspeccionar evidencia durable y reconstruir qué producto se desarrolla, qué decisiones siguen vigentes, qué versión del código y del estado técnico es actual, qué trabajo está autorizado, qué se comprobó realmente y cuál es la próxima acción válida. Los chats pueden terminar, los servicios pueden cambiar y un agente puede perder contexto. El proyecto no debería exigir que alguien recuerde toda la conversación anterior.

En los ejemplos se usan Windows, GitHub, Python y agentes conversacionales porque son un punto de partida conocido. No son dependencias universales: se puede trabajar en otro sistema operativo, con otra pila tecnológica o exclusivamente con personas. Los modelos externos, APIs pagas, credenciales y servicios de nube son opciones separadas, que requieren configuración y autorización específica.

Hay tres afirmaciones que nunca conviene confundir:

- El starter puede instalarse y sus mecanismos portables pasan sus propias comprobaciones.
- Tu aplicación adoptó el starter mediante requisitos, autoridades y estado actuales coherentes.
- Tu aplicación cumple sus funciones, seguridad, confiabilidad y expectativas de uso.

La primera afirmación no demuestra las otras dos. Una instalación puede estar correctamente en `HOLD / UNTRUSTED_CONTEXT` sin que eso signifique que haya fallado; significa que todavía falta reconstruir y confirmar el proyecto real.

2. Quién toma cada decisión

La arquitectura distribuye responsabilidades, no impone una cantidad fija de personas, modelos o ventanas de chat. Un proyecto pequeño puede funcionar con una persona dueña del producto, un rol de planificación y un ejecutor. Separar un Master Owner cobra sentido cuando existen productos diferentes, recursos compartidos, políticas transversales o fronteras de coordinación reales. Abrir otra conversación con el título «Master Owner» no crea por sí mismo esa necesidad.

Participante	Responsabilidad principal	Qué no debería resolver rutinariamente
CLIENT / dueño del producto	Problema, usuarios, prioridades, reglas de negocio, experiencia esperada, presupuesto, publicación y demás autorizaciones reservadas.	Conflictos de Git, esquemas internos, estrategia de tests o reparación de currentness.
I1 · Master Owner, cuando corresponde	Coherencia entre productos, recursos compartidos, problemas sistémicos, capacidad y fronteras de ownership.	Sustituir al Planner o coordinar cada pequeña tarea.

Participante	Responsabilidad principal	Qué no debería resolver rutinariamente
I2 · Planner	Interpretación del producto, estrategia, arquitectura, límites de capacidades, charters y auditoría estratégica.	Convertirse en el bucle de depuración o escribir un nuevo charter por cada bug.
I3 · Execution Controller / Campaign Lead	Implementación, integración, reintentos, depuración, revisión independiente, checkpoints y recuperación dentro de la campaña autorizada.	Inventar reglas de negocio o aprobar unilateralmente su resultado final.
I4 · Especialistas de ejecución	Código, investigación acotada, pruebas, revisión técnica y evidencia vinculada a la fuente.	Reemplazar decisiones del CLIENT o declarar aceptación estratégica.

El dueño del producto puede rechazar una interfaz que resulta confusa aunque las pruebas automáticas sean verdes. Un revisor técnico puede rechazar un candidato que no conserva los datos. El Planner evalúa si la capacidad cumple las condiciones estratégicas del charter. Son decisiones relacionadas, pero una no reemplaza silenciosamente a otra.

En un flujo posible, ChatGPT ayuda a planificar y Codex modifica el repositorio. Es un ejemplo, no una obligación de utilizar una plataforma ni una suscripción. El rol depende de su autoridad y de las evidencias que debe producir. Una misma persona puede cubrir varias responsabilidades acotadas, siempre que se respeten los conflictos de interés y las necesidades reales de revisión independiente.

3. La información que debe permanecer accesible

Para averiguar el estado técnico, empezá por el código, los artefactos y el entorno que realmente existen. Contrastalos con el Product State canónico, los requisitos actuales, la autoridad de campaña y, cuando importa, las evidencias de DB y runtime. Un resumen viejo del chat, aun si dice «PASS», no tiene más autoridad que la evidencia física vigente.

Una nueva decisión explícita del CLIENT puede cambiar el producto. El Planner debe registrarla como requisito actual y, cuando corresponde, marcar la decisión anterior como sustituida. No alcanza con elegir el archivo que parece más reciente por el nombre. Si falta un hecho, se marca `UNKNOWN`. Si dos autoridades actuales se contradicen materialmente, corresponde `HOLD` para las acciones dependientes hasta reconciliar el conflicto. Las investigaciones de solo lectura pueden continuar.

El sistema distingue tres planos de continuidad:

1. L0: evidencia de lo que el CLIENT solicitó, corrigió y aprobó, incluidas las decisiones posteriores.
2. L1: requisitos interpretados, teoría de producto y Product State canónicos vigentes.

3. L2: implementación, código, bases de datos, campañas, checkpoints, revisiones y pruebas de runtime.

La separación evita que una promesa formulada en una conversación se confunda con software desplegado. También impide que una propuesta antigua reemplace una implementación posteriormente comprobada sin registrar esa decisión.

Como mínimo, un trabajador que llega a un contexto nuevo debe reconstruir el objetivo, los requisitos, el ref de código que puede comprobar, los testigos relevantes del entorno o de la DB, la campaña activa si existe, el gate abierto, la mejor evidencia disponible, las incertidumbres y la próxima acción permitida o prohibida. Si no puede hacerlo, todavía no se demostró continuidad.

Los términos que aparecen a lo largo de los ocho capítulos son:

- **Scaffold:** la carpeta de control `.agentic-sdlc/` instalada en la aplicación. Comienza deliberadamente en `HOLD / UNTRUSTED_CONTEXT`.
- **Discovery:** investigación inicial, preferentemente de solo lectura, para conocer código, uso, entorno, datos y necesidades reales.
- **Adoption:** promoción controlada de hechos verificados y decisiones confirmadas a un estado coherente. No es copiar una plantilla sin revisar.
- **Product State:** referencia canónica del estado actual de las capacidades del producto.
- **Charter:** contrato estratégico de una capacidad, sus límites, dependencias, gates y falsificadores.
- **Gate:** condición de verificación antes de una aceptación; distintos riesgos exigen distintas pruebas.
- **Currentness:** comprobación de que código, requisitos, estado, autoridad y artefactos siguen vinculados a sus versiones reales.

4. Cómo leer el manual completo

El capítulo 01 conviene leerlo antes de la primera instalación. El 02 corresponde antes de autorizar la primera función material. Los demás se consultan cuando aparece su problema operativo. No hace falta memorizar unas sesenta páginas para probar el perfil `SMALL` en un proyecto nuevo y sin gobierno previo.

Capítulo	Cuándo consultarlo	Resultado que deberías poder comprobar
00 · Antes de empezar	Antes de instalar.	Entendés las responsabilidades, límites y recorrido.
01 · Instalación	Al preparar un proyecto.	Existe un control layer validado, sin inventar adopción ni permiso de ejecución.
02 · Planner y adopción	Antes de la primera capacidad.	Requisitos, baseline, autoridad y próximo paso se reconstruyen sin el chat anterior.

Capítulo	Cuándo consultarlo	Resultado que deberías poder comprobar
03 · Operación diaria	Durante el desarrollo.	El trabajo se autoriza, ejecuta, verifica y registra con continuidad en la misma campaña.
04 · Solicitar cambios	Al definir funciones o revisar decisiones.	Cada pedido describe comportamiento, prioridades y límites observables.
05 · Errores y recuperación	Ante incidentes y dudas de estado.	Se conserva evidencia y se evita daño adicional hasta verificar la recuperación.
06 · Cierre de capacidades	Cuando alguien dice «terminado».	El candidato exacto posee evidencia suficiente para los gates que le corresponden.
07 · Ejemplo guiado del club	Para practicar el flujo integral.	Podés seguir una capacidad desde requisitos hasta aceptación.

Después del 01 puede ser correcto seguir en HOLD. Después del 02, una sesión genuinamente nueva debería poder reconstruir el proyecto. Después del 06, el candidato aceptado debería tener identidad y evidencias separadas por gate. Si falta alguno de esos resultados, hay que volver a producir la evidencia correspondiente, no repetir una explicación más larga.

5. Preparación: las decisiones que sí son tuyas

Antes de instalar, prepará una ficha breve. Describí el problema, las personas usuarias, el recorrido principal, los datos, el contexto de uso, el plazo, el presupuesto, las expectativas de interfaz y las exclusiones. Si una decisión todavía no se tomó, señalala como pregunta abierta.

Por ejemplo:

> Quiero una aplicación local para registrar socios y cuotas de un club. La usa una persona administrativa. La primera versión no acepta pagos online. Debe conservar el historial de pagos. Prefiero una pantalla simple y legible. El presupuesto de servicios pagos es cero.

No necesitás elegir SQLite o PostgreSQL, determinar la cantidad de ramas de Git, seleccionar un ORM ni programar un rollback. Esas decisiones corresponden a ingeniería, que debe explicarte las consecuencias cuando afectan el producto o un permiso reservado. Cuando hay datos personales, pagos, publicación o conexiones externas, los agentes no eliminan tus responsabilidades de autorización ni las obligaciones legales pertinentes.

6. Alcance, límites y versiones

ArquitecturaRepo distribuye mecanismos reutilizables y ejemplos sintéticos. Sus pruebas de transferencia y recuperación no certifican automáticamente que cada aplicación se pueda instalar, operar o publicar con seguridad. `tools/campaignctl.py` ofrece un runtime de referencia opcional; por sí solo no llama a cualquier agente de programación ni certifica la integración nativa con Codex Desktop.

La versión v1.4 distingue controles portables comprobados, mecanismos experimentales y capacidades de hosts externos no verificadas. Para un uso real hay que contrastar las instrucciones sensibles a versión con el código y la release que efectivamente se instalaron. Precios, opciones de modelos, pantallas de servicios y restricciones de terceros pueden cambiar; no son invariantes del método.

El código y la documentación originales del proyecto tienen licencia MIT; las dependencias y los recursos ajenos pueden tener sus propias condiciones. El tag identifica una versión, pero la integridad del archivo distribuido requiere comprobar su checksum. Que exista una release verificada no autoriza una campaña nueva, una compra de API ni un despliegue público de tu aplicación.

7. Un recorrido de extremo a extremo

Imaginá una asociación que necesita registrar socios. El CLIENT decide quién puede dar de alta una ficha, qué datos son obligatorios y qué significa desactivarla. El Planner convierte esas decisiones en requisitos verificables; no supone que desactivar sea borrar para siempre. Antes de ordenar implementación, inspecciona lo que ya existe, identifica dependencias y consulta únicamente las decisiones que cambian el comportamiento esperado.

Tras confirmar el alcance, el Planner persiste requisitos actuales, Product State, baseline técnico y plan de entrega con fechas. Elige una capacidad que pueda demostrarse de principio a fin. Su charter expresa el resultado, el motivo, las fronteras permitidas, las interfaces afectadas, los gates y las condiciones que invalidarían el encargo.

I3 organiza la implementación. Puede encargar código, pruebas o review a especialistas o resolverlos directamente en un proyecto pequeño. Repara bugs ordinarios y vuelve a correr las verificaciones aplicables dentro de la misma campaña. No devuelve cada fallo al Planner como un nuevo problema estratégico. Cuando hay un candidato significativo, el Planner inspecciona la evidencia y el CLIENT evalúa los aspectos de producto y experiencia que le corresponden.

El ciclo no termina porque el ejecutor escribió un informe convincente ni porque hubo muchos commits. Termina cuando la capacidad funciona con el nivel aceptado y el repositorio preserva suficientes hechos actuales para que otro trabajador continúe. El capítulo 07 recorre este ejemplo con datos ficticios.

8. Decisiones habituales y sus responsables

Situación	Responsable y evidencia esperada
«Los socios con cuotas vencidas no pueden inscribirse».	CLIENT confirma la regla; Planner la registra en los requisitos vigentes.

Situación	Responsable y evidencia esperada
«¿Usamos SQLite o PostgreSQL?»	Ingeniería elige según carga real, concurrencia, backups, despliegue y riesgos.
«¿Podemos mostrar datos de socios reales en una demo?»	Hace falta autorización lícita, concreta y controles de acceso; lo normal es usar datos ficticios.
«Una prueba falló después de una corrección».	I3 investiga, corrige y renueva evidencia dentro del mismo charter.
«Ahora quiero transformar la app local en un servicio público».	CLIENT confirma el cambio; Planner reconsidera red, seguridad, costo y alcance.
«El agente dijo que todo está listo».	El mensaje no constituye aceptación; hay que identificar el candidato y sus pruebas técnicas y humanas.

9. Ficha de arranque de una página

Podés empezar sin una especificación técnica extensa. Una ficha útil contiene:

- **Propósito:** qué problema resuelve la primera versión y para quién.
- **Recorrido principal:** qué intenta lograr la persona y cómo se ve el éxito.
- **Datos:** qué se almacena, si hay información personal y cuánto debe conservarse.
- **Entorno:** equipos, ubicación, conectividad y funcionamiento sin Internet.
- **Límites:** fecha objetivo, presupuesto, integraciones aprobadas y funciones excluidas.
- **Calidad visible:** legibilidad, accesibilidad, errores comprensibles y rendimiento que importan al usuario.

No hace falta que resuelvas todas las incógnitas de arquitectura. El sistema de ingeniería debe investigar lo que pueda y exponerte solamente las decisiones de producto que no debe inventar. La ficha sirve de entrada al discovery y a la adopción del capítulo 02.

10. Cómo evaluar si el sistema te está ayudando

Tras dos o tres ciclos de entrega, pedí una demostración breve: ¿qué capacidad nueva puedo usar?, ¿dónde está la evidencia?, ¿qué cambió desde el ciclo anterior?, ¿qué riesgo material sigue abierto?, ¿cuál es la próxima acción autorizada? Si la respuesta se limita a «hicimos muchos commits», «abrimos varios agentes» o «redactamos más documentación», todavía no quedó demostrado el avance del producto.

Una buena arquitectura puede reducir la dependencia de conversaciones anteriores, pero no reemplaza el juicio humano, la revisión de seguridad o la prueba de funcionamiento real. Medí su valor por continuidad, claridad de responsabilidades, capacidades utilizables, calidad de evidencia y recuperación confiable; no por la cantidad de protocolos, sesiones o pantallas.

Siguiente: capítulo 01, Instalación y configuración.

Fuentes y vigencia

- Release pública v1.4.0
- Primera instalación y límites de soporte
- Notas de release y actualización
- Visión general de arquitectura
- Modelo de autoridad
- Licencia MIT

Este capítulo describe el starter y un ejemplo ficticio. Ninguna prueba genérica sustituye la comprobación del proyecto real que lo adopte.

01 — Instalación y configuración

ArquitecturaRepo · Manual completo de uso · Edición v1.4 · Español

> Este capítulo explica la primera instalación y su separación de la adopción del proyecto real. Los comandos siguen la guía pública v1.4.0. Un scaffold creado correctamente **no** significa que la aplicación ya fue comprendida, adoptada, implementada o aceptada.

Contenido del capítulo

1. Entorno y cuentas
2. Starter, fork y aplicación: tres ubicaciones distintas
3. Verificar rutas antes de escribir
4. Elegir un perfil proporcional
5. Crear el control layer
6. Validar sin inventar aceptación
7. La próxima acción es discovery, no programar
8. Antes de repetir una instalación
9. Del scaffold a una adopción real
10. Conexiones, permisos y privacidad
11. Diagnóstico seguro

1. Entorno y cuentas

Este recorrido utiliza Windows, PowerShell, Python y una copia local de ArquitecturaRepo. GitHub permite alojar el código, un asistente conversacional puede ayudar con planificación y un agente de programación puede trabajar en una carpeta autorizada. Ninguno de esos servicios comerciales es una dependencia universal del sistema. Si manejás Git por consola, no necesitás GitHub Desktop. Para usar agentes remotos tampoco necesitás una GPU dedicada en tu computadora.

Lo que sí necesitás es una ubicación de trabajo identificable, suficiente espacio en disco, acceso confiable a los archivos y permisos para instalar las herramientas elegidas. Antes de autorizar escrituras, comprobá qué repositorio contiene la aplicación que querés desarrollar y qué carpeta puede modificar el ejecutor. Un error común es abrir el starter genérico y empezar a programar ahí una función de una aplicación distinta. El starter y la aplicación tienen finalidades separadas y normalmente viven en repositorios diferentes.

Desde PowerShell, comprobá la disponibilidad de Python:

```
& "python" --version
& "py" --version
```

Python 3.12 es la versión comprobada por la CI portable de v1.4. Si ninguno de los comandos funciona, instalá una versión compatible desde su distribuidor oficial, reiniciá la terminal y repetí la verificación. No uses instaladores de procedencia incierta. Si un comando termina con error, no supongas que tuvo éxito porque no mostró una ventana nueva. Las interfaces de GitHub, ChatGPT y los editores pueden

cambiar; buscá la función necesaria y no dependas de la ubicación exacta de un botón.

Protegé tus cuentas con autenticación adecuada y permisos mínimos. No pegues contraseñas, claves privadas, tokens ni cadenas de conexión en prompts, commits o archivos públicos. Conectar un proveedor a ChatGPT no demuestra que el Planner haya visto el repo correcto, ni que el servicio tenga permiso para escribir sobre tu aplicación. La existencia de una herramienta y la autorización para usarla son asuntos distintos.

2. Starter, fork y aplicación: tres ubicaciones distintas

Ubicación	Finalidad	Confusión que hay que evitar
`ArquitecturaRepo/`	Código público de referencia, protocolos, plantillas, scripts y tests para crear un control layer.	No contiene por defecto tu aplicación ni su estado vigente.
Tu fork opcional	Copia propia del starter para mantener modificaciones o adaptaciones.	No equivale a crear un producto nuevo.
`club-app/` o el proyecto real	Código, datos, historial y control `.agentic-sdlc/` de la aplicación concreta.	No es un ejemplo descartable del framework.

Los ejemplos del starter utilizan hechos sintéticos. Copiar su Product State a una aplicación real afirmaría requisitos, ref de código, ownership y decisiones que quizás nunca existieron. Por eso la instalación empieza de forma prudente, sin marcar automáticamente un estado adoptado.

Una organización sencilla de carpetas locales es:

```
C:\Proyectos\
    ArquitecturaRepo\
    club-app\
```

Revisá el README y la licencia MIT. Si querés adaptar el framework de forma independiente, podés mantener un fork; si no, alcanza con un checkout normal. La aplicación puede ser privada. Si ya tiene código, datos o historial, utilizá ese repositorio real en lugar de inventar otro «desde cero» para imitar el manual.

Para obtener una versión reproducible, elegí el release v1.4.0, que incluye ZIP portable, checksum y manifest. Verificá los bytes descargados. El branch `main` puede evolucionar con documentación nueva después de publicarse el paquete; lo que aparece en GitHub hoy no debe atribuirse retroactivamente al ZIP de la release.

3. Verificar rutas antes de escribir

Abrí PowerShell en la copia local del starter y ejecutá comandos de inspección:

```
Get-Location
& "python" --version
& "git" status --short
```

Si Git informa que no estás dentro de un repositorio, revisá la ubicación. Si aparecen archivos modificados, no los descartes para obtener un árbol «limpio» sin investigar a quién pertenecen. Comprobá el destino que vas a pasar al instalador y no confundas la ruta actual de la terminal con la ruta donde querés crear `.agentic-sdlc/`.

Si utilizás GitHub Desktop, verificá qué repositorio está activo. Si utilizás Codex u otro agente, abrí la carpeta exacta de la aplicación y proporcioná la ubicación del starter como referencia de lectura hasta aclarar los permisos de instalación. Un agente autorizado puede realizar la adaptación, pero primero debe inspeccionar el destino y detectar si ya existe gobierno del proyecto.

Si encontrás un `.agentic-sdlc/` anterior, una campaña activa, Product State, historial real o una base de datos, detené la instalación genérica. No borres el proyecto para seguir el tutorial. Para una aplicación ya gobernada corresponde el preflight de attachment de solo lectura. Ese proceso preserva referencias y puede comprobar currentness, pero **no** es un actualizador automático de v1.2/v1.3 ni autoriza ejecución nueva.

4. Elegir un perfil proporcional

Los perfiles responden a riesgos y características del producto, no a cuántos agentes querés utilizar. Las seis familias están definidas en profiles/profiles.json:

Perfil	Cuándo considerarlo
`SMALL`	Un producto pequeño sin persistencia material ni riesgos que justifiquen controles adicionales.
`STATEFUL`	Datos persistentes, usuarios, cambios de esquema, recuperación, backups o runtime con estado.
`ARTIFACT_HEAVY`	Archivos numerosos o costosos, lineage, almacenamiento, ciclo de vida o derechos de uso.
`MULTI_WORKSTREAM`	Varias líneas de trabajo planificadas de forma independiente para un único producto final.
`MULTI_PRODUCT`	Productos distintos con recursos, políticas o infraestructura compartidos; puede justificar I1.
`HIGH_CONSEQUENCE`	Consecuencias materiales sobre privacidad, seguridad, integridad, finanzas o acciones irreversibles.

`HIGH_CONSEQUENCE` no es una medalla de profesionalismo y `MULTI_PRODUCT` no obliga a contratar más servicios o multiplicar conversaciones. El perfil selecciona familias de controles; no crea hechos del proyecto ni permisos para publicar, gastar o procesar datos. Documentá qué características justifican la elección y revisala cuando cambien los riesgos verificados.

La aplicación ficticia del club se clasificaría razonablemente como `STATEFUL` porque conserva socios, cuotas y actividades. No necesita necesariamente un Master Owner separado ni una flota de agentes. Para un primer experimento realmente nuevo y sin gobierno previo, `SMALL` suele ser una base más proporcional. Elegí los mecanismos necesarios para el problema, no la arquitectura más grande disponible.

5. Crear el control layer

Desde la carpeta del starter v1.4, en PowerShell, podés crear un control para un proyecto nuevo. Revisá primero el destino:

```
& "python" "tools\scaffold_project.py" --target "..\mi-proyecto" --profile "SMALL" --p
```

Si se trata del club ficticio y el perfil `STATEFUL` está justificado:

```
& "python" "tools\scaffold_project.py" --target "..\club-app" --profile "STATEFUL" --p
```

Estos comandos son ejemplos de una sola línea; cambiá la ruta, el identificador y el objetivo según el proyecto concreto. El script crea `.agentic-sdlc/` dentro de la aplicación sin reescribir por defecto su código o sus workflows. Debe rechazar sobreescrituras de gobierno canónico existente. No uses opciones de fuerza como método rutinario de reparación.

El estado inicial esperado es `HOLD / UNTRUSTED_CONTEXT`. Significa que el framework no inventó los requisitos, los owners, el ref del código, la campaña ni la aceptación del producto. Es un resultado correcto. Cambiar manualmente el JSON a `READY` sería falsear la continuidad; todavía falta discovery y adopción.

El instalador puede ofrecer `--with-runtime` para agregar maquinaria de referencia de campañas. Esa opción no autentica proveedores, no inicia Codex Desktop y no crea permiso para ejecutar cualquier instrucción. La opción `--no-feedback-workflow` evita instalar el programador periódico de feedback, pero **no** desactiva por sí sola el generador local de metadatos. Su configuración es una decisión separada.

6. Validar sin inventar aceptación

Una vez creado el scaffold, entrá en la carpeta de la aplicación y usá los comandos vigentes de v1.4:

```
& "python" -m pip install -r ".agentic-sdlc\requirements-validation.txt"
& "python" ".agentic-sdlc\tools\project_status.py" --control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" --project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-sdlc"
```

El primer comando instala dependencias locales de validación. `project_status.py` permite inspeccionar el estado inicial; `discover_project.py` inventaría metadatos y hashes de archivos sin copiar su contenido a un servicio remoto; `reconstruct_context.py` comprueba qué estado puede recuperar un trabajador nuevo desde las superficies durables. La discovery rechaza enlaces a archivos externos a la carpeta objetivo y destinos vinculados que no cumplen su contrato.

Es posible que la reconstrucción termine en HOLD porque aún no se adoptó un baseline confirmado. Ese estado puede ser el resultado correcto: la herramienta debe explicar qué evidencia falta, no inventarla. Los validadores de contratos no comprueban por sí solos que la aplicación abra, que una cuota se guarde en una DB real o que el usuario entienda la interfaz.

Verificá tres hechos: existe el control layer en la ruta prevista; los archivos y workflows anteriores de la aplicación permanecen intactos; los resultados de estado, inventario y reconstrucción son accesibles sin depender del chat. Conservá los mensajes de error. Si falta `jsonschema`, instalá las dependencias declaradas en vez de editar Product State o recrear el repositorio.

El resultado de esta etapa es **control inicial instalado e inspeccionado**, no «aplicación adoptada». La adopción exige evidencias del proyecto real y la confirmación de los requisitos por su dueño.

7. La próxima acción es discovery, no programar

Seguí las instrucciones de instalación y adaptación, la sesión de descubrimiento del producto y el contrato de transferencia y adopción. Para una aplicación anterior sin el mismo baseline también puede importar el prompt de reanclaje.

El Planner identifica qué hace el código actual, qué quiere cambiar el CLIENT, cuáles son los datos y el entorno relevantes, quién tiene autoridad y qué preguntas siguen abiertas. Una campaña material debe esperar un baseline adoptado y un charter explícito. Programar apenas aparece `.agentic-sdlc/` saltearía el paso que conecta el sistema con la verdad del proyecto.

8. Antes de repetir una instalación

Si `.agentic-sdlc/` ya existe, investigá su contenido antes de reintentar. Puede conservar el Product State canónico, ownership, una campaña vigente, referencias a la DB, lease o evidencia imprescindible. Una segunda instalación que parece funcionar puede destruir continuidad si reemplaza esas superficies.

Ante un error, no empieces por borrar, forzar ni reclonar. Clasificá si el problema se debe a entorno, ruta, dependencia, contrato, versión, currentness o control canónico preexistente. Conservá la salida completa y dirigí la corrección a su owner técnico. Un repositorio ya gobernado requiere attachment read-only y una migración evaluada por separado, no el scaffold genérico.

9. Del scaffold a una adopción real

Una secuencia razonable de adopción tiene pasos distintos:

1. Investigar aplicación y entorno reales, inicialmente de solo lectura.
2. Confirmar con el CLIENT los requisitos y exclusiones actuales.
3. Registrar un baseline de producto y un baseline técnico vinculados al código comprobado.
4. Elaborar un plan con hitos, fechas y condiciones observables de terminado.
5. Preparar una especificación explícita de adopción y comprobar preflight.
6. Promover registros de estado y ownership de manera coherente y recuperable.
7. Validar el resultado y demostrar reconstrucción desde contexto fresco.
8. Autorizar recién entonces una capacidad material mediante su charter.

La guía de adopción detalla esa transición. Un agente no debería inventar un JSON, un SHA, un Goal activo ni una aprobación para obtener un informe verde. Cuando falta una decisión de producto, responde el CLIENT. Cuando falta evidencia técnica, el

dueño de ingeniería investiga y demuestra el resultado. El CLIENT puede actuar como interlocutor y aprobador sin tener que reconciliar a mano los registros internos.

10. Conexiones, permisos y privacidad

Podés conectar GitHub u otro servicio para facilitar lecturas. Concedé el alcance mínimo necesario y verificá la rama y el repositorio concretos. Una sesión autenticada no prueba que el agente vea toda la evidencia o que tenga el permiso correcto. Antes de escrituras, revisá carpeta, worktree, permisos y charter aplicable.

Las comprobaciones locales reversibles son distintas de publicar, desplegar, enviar mensajes, comprar llamadas de API, borrar datos o compartir información privada. Estos efectos requieren autorización específica, alcance definido y una evaluación proporcional. No guardes secretos en prompts o repositorios públicos. Una aplicación construida con este starter puede conservar su código y datos privados.

En v1.4 la configuración del generador local de feedback de metadatos estrictos está habilitada por defecto; no se incluye un collector externo en el paquete público. El parámetro `--no-feedback-workflow` desactiva el programador de GitHub Actions, no el generador local. Para desactivar completamente la generación, establecé `enabled=false` en la configuración instalada. Consultá la política de privacidad y feedback antes de habilitar cualquier envío o aceptar cambios en workflows ejecutables.

11. Diagnóstico seguro

Observación	Interpretación prudente	Próxima comprobación
Python no se reconoce.	La terminal no localiza el intérprete.	Revisar instalación y ruta; no tocar la app.
Falta `.agentic-sdlc/`.	La creación falló o apuntó a otra carpeta.	Comparar ruta objetivo y salida del instalador.
`HOLD / UNTRUSTED_CONTEXT`.	Hay scaffold pero faltan hechos confirmados del producto.	Seguir con discovery y adopción.
Falta `jsonschema`.	No se instalaron las dependencias del intérprete actual.	Instalar requirements y volver a inspeccionar.
Se preservó un control existente.	El instalador protegió gobierno canónico.	Usar el preflight read-only correspondiente.
Falló validación o reconstrucción.	Puede existir conflicto estructural, de fuente o autoridad.	Conservar el informe y reparar la causa comprobada.
El agente ve otro repositorio.	Su alcance operativo apunta a otra carpeta.	Detener escrituras y confirmar worktree y permisos.
Tests verdes pero la app no abre.	Las pruebas del control no cubrieron comportamiento físico.	Ejecutar la aplicación real y probar el recorrido del usuario.

El cierre de este capítulo es un control layer inicial cuyo estado y límites se pueden

inspeccionar. Su adaptación a requisitos, código, DB y autoridad reales constituye un trabajo distinto. El capítulo 02 explica esa frontera.

Siguiente: capítulo 02, El Planner y la adopción.

Fuentes y vigencia

- GitHub Release v1.4.0
- Inicio público y soporte comprobado
- Notas de release y actualización
- Fork y bootstrap
- Transferencia y adopción
- Attachment de proyectos gobernados
- Validación ejecutable
- Privacidad del feedback
- Código del instalador

Las instrucciones corresponden al starter o a ejemplos ficticios. No prueban que un proyecto concreto esté instalado, adoptado o aceptado.

02 — El Planner y la adopción del proyecto

ArquitecturaRepo · Manual completo de uso · Edición v1.4 · Español

> Crear `.agentic-sdlc/` es instalar una capa inicial, no confirmar el baseline de una aplicación. Este capítulo explica cómo el Planner reconstruye el proyecto real, obtiene decisiones del CLIENT, adopta estado verificado y autoriza la primera campaña. Para comandos concretos, consultá la guía pública v1.4 y el contrato de adopción.

Contenido del capítulo

1. Una conversación no es el proyecto
2. Preparar el espacio de trabajo
3. Discovery: hacer las preguntas correctas
4. Baseline técnico y plan de entrega
5. Adoption: conectar el starter con el proyecto real
6. El examen decisivo: contexto fresco
7. Primera campaña autorizada
8. Ejemplo de sesión inicial con el Planner
9. Un baseline que el CLIENT pueda confirmar
10. Ejemplo de primera campaña

1. Una conversación no es el proyecto

El I2 Planner es responsable de la estrategia de un producto o workstream delimitado. Puede trabajar mediante una conversación, pero el historial del chat no es una base de datos canónica ni prueba que la última implementación coincida con una afirmación anterior. Su autoridad proviene de los requisitos actuales confirmados por el CLIENT, los protocolos aplicables, la evidencia física del repositorio y de la DB, el ownership vigente y los registros válidos de campaña.

Si el último mensaje del ejecutor dice «terminado», el Planner todavía tiene que identificar el candidato exacto, comprobar la evidencia y distinguir qué gates realmente pasaron. No puede declarar aceptación estratégica porque el informe tenga una etiqueta `PASS` o porque un agente asegura que está todo resuelto.

En un proyecto pequeño, un Planner y un dueño de ejecución pueden ser suficientes. La figura I1 Master Owner tiene sentido cuando hay varios productos, líneas de trabajo independientes, recursos o presupuestos compartidos, o una necesidad sistémica de coordinar políticas. No reemplaza al Planner del producto y no mejora el sistema por existir como un chat adicional.

El Planner piensa desde el resultado final: quién utilizará la capacidad, qué interfaces y datos actuales afecta, cómo se observará el éxito, cuáles son las dependencias comprobadas y qué hallazgos impedirían aceptar el resultado. Con esa información prepara un charter de capacidad. Dentro del alcance autorizado, I3 se encarga de programación, depuración, reintentos, integración, revisión y recuperación. Un defecto ordinario no exige un nuevo charter ni una nueva campaña.

2. Preparar el espacio de trabajo

Empezá por el repositorio real de la aplicación, no por un resumen recordado. Leé el `AGENTS.md` vigente, los protocolos aplicables, los requisitos, el Product State canónico, las referencias actuales de código y las campañas o registros de runtime relevantes. Si el Planner no puede leer una superficie necesaria, debe identificar la falta concreta. No corresponde completar el hueco con memoria de una conversación distinta.

Podés organizar el trabajo en un Proyecto de ChatGPT u otro espacio de colaboración y guardar ahí el manual como referencia. Si conectás GitHub, limitá permisos y repositorios al alcance necesario. Verificá la lectura de un archivo conocido en el ref correcto. Que la cuenta aparezca conectada no significa que el asistente tenga acceso a todas las ramas, issues, bases de datos o procesos de ejecución. Ningún modelo, configuración de esfuerzo o suscripción específica es requisito universal del framework.

Una instrucción inicial útil para un Planner es:

> Actuá como I2 Planner de este proyecto. Yo soy CLIENT: defino reglas de negocio, prioridades, requisitos y autorizaciones externas reservadas. Antes de planificar, reconstruí el repositorio real y los protocolos aplicables; no inventes estado. Definí límites de capacidades y charters. Dejá implementación, depuración y recuperación táctica en I3, dentro del alcance autorizado. No me transfieras decisiones rutinarias de Git, pruebas, currentness o debugging. Distinguí lo que está comprobado, lo que sigue desconocido y las decisiones de producto que realmente necesito tomar.

El texto es una entrada de rol, no sustituye al `AGENTS.md` real, al registro de requisitos ni a una campaña que ya tiene autoridad. En cada nuevo contexto hay que reconstruir las superficies vigentes. Un título de rol tampoco concede permisos adicionales de filesystem o proveedores externos.

3. Discovery: hacer las preguntas correctas

El discovery comienza por la persona y el problema, no por elegir framework. Preguntá qué usuario necesita completar qué recorrido, cuáles son los datos persistentes, los errores importantes, la conectividad, la exposición a Internet, la sensibilidad de los datos, los costos, las fechas, los requisitos de accesibilidad y las exclusiones. El Planner puede agrupar tres a siete preguntas relacionadas y luego devolver una síntesis coherente para que el CLIENT la corrija.

La pregunta adecuada modifica una decisión de producto:

Pregunta útil para el CLIENT	Decisión técnica que normalmente pertenece a ingeniería
¿Quién utiliza el sistema, en qué equipos y desde dónde?	Puertos, topología, framework de interfaz y despliegue.
¿Qué sucede con los pagos cuando se desactiva un socio?	Tablas, claves foráneas, migraciones y transacciones.
¿Se permitirán pagos por Internet en la primera versión?	Diseño del proveedor, respetando autorización de gasto y operaciones reales.
¿Qué resultado debe poder demostrarse antes de la fecha objetivo?	Orden interno de tareas, mecanismos de pruebas y checkpoints.

Pregunta útil para el CLIENT	Decisión técnica que normalmente pertenece a ingeniería
¿Varias personas editarán la misma ficha simultáneamente?	Concurrencia, transacciones y resolución de conflictos.

Al cerrar discovery, presentá una sola revisión del baseline: funciones, recorrido, reglas de datos, calidad no funcional, seguridad y red, comportamiento de interfaz, backups, recuperación, presupuesto, fechas, exclusiones y preguntas pendientes. El CLIENT confirma si esa síntesis representa el producto actual. La confirmación no es un contrato legal ni impide cambios futuros; fija una versión rastreable de la decisión que vale para planificar ahora.

No descargues sobre el CLIENT incertidumbres exclusivamente técnicas. «¿Qué base de datos elegimos?» suele ser una pregunta de ingeniería. «¿Cuántas horas de pagos es aceptable perder después de un corte de luz?» es una pregunta de producto porque condiciona la estrategia de respaldo. El Planner debe explicar la consecuencia para la persona usuaria antes de pedir una decisión reservada.

4. Baseline técnico y plan de entrega

Cuando la intención del producto está confirmada, el Planner elige o confirma la estrategia técnica contra el código y las restricciones reales. Registra lenguaje, frameworks, persistencia, migraciones, despliegue, secretos, permisos, pruebas, integración de UI, backups y recuperación. En el control layer instalado, las decisiones vigentes se conservan en `state/TECHNICAL_BASELINE_CURRENT.json`, no únicamente en un comentario de chat.

El plan de entrega describe hitos observables, fechas objetivo, dependencias, confianza, riesgos y próxima revisión. Si una fecha cambia, debe quedar registrado el cambio. Sobrescribir silenciosamente la fecha anterior destruye información importante para razonar sobre el progreso y los compromisos. Una lista de tareas de programación sin plazos ni resultados visibles no sustituye el plan de entrega.

`tools/baseline_guard.py` verifica si las precondiciones de una campaña material están coherentes. No debería eludirse mediante campos inventados. Si los requisitos no están confirmados, hay una decisión material de seguridad/red pendiente o las referencias actuales se contradicen, se mantiene HOLD para la campaña dependiente.

El Planner también debe comprobar que el encargo sea ejecutable. Un charter que solo dice «implementar gestión de socios» puede definir una idea estratégica, pero resultar demasiado abstracto para un controlador que hará muchas operaciones o consumirá recursos. El contrato de ejecución orientada al resultado propone un work order acotado cuando hay múltiples operaciones materiales, lotes, proveedores pagos o runtime significativo.

El charter define qué y por qué; el work order vincula inventario real, operaciones viables, presupuesto, resultado físico, límites y condiciones de parada. No es una segunda campaña. `tools/work_order_guard.py` permite ejecutar `validate` sobre la estructura y `assess` sobre recibos de avance, pero su evaluación no constituye aceptación física, perceptual ni autorización externa. Si una operación se resuelve con herramientas deterministas del sistema operativo, no hay motivo para imponer una consulta paga a un modelo por cada archivo.

5. Adoption: conectar el starter con el proyecto real

La adopción debe describir la aplicación que existe físicamente. El recorrido reproducible comienza con scaffold, inventario, especificación de adopción revisada, preflight sobre una copia temporal del control y promoción de registros con journal. `discover_project.py` enumera metadatos y hashes; no deduce por sí solo los requisitos de negocio. `apply_adoption.py` materializa hechos verificados y decisiones expresadas en una especificación válida, pero no puede inventar la intención del CLIENT.

El resultado adoptado vincula requisitos, fuentes de código, baseline técnico, Product State, contexto de entrada, registro de owners y overlays del proyecto. Cuando una base de datos o un testigo SQLite WAL es relevante, tiene que concordar con las referencias actuales. Si ya hay una campaña activa, no se debe usar la adopción como un reinicio disimulado. En una aplicación con control canónico previo, el preflight read-only es el camino inicial, no una reinstalación genérica.

Si el Planner solo puede leer el repo remotamente y el I3 tiene un checkout local autorizado, el Planner prepara un despacho completo. No le pide al CLIENT que edite manualmente registros de ownership, concilie SHAs o ensamble fragmentos de JSON. El ejecutor utiliza la evidencia real y reporta resultados físicos. El código, los datos y el historial previos se conservan salvo una modificación específica, justificada y autorizada.

El informe de adopción debería responder cinco preguntas:

1. ¿Qué repositorio y versión del código se inspeccionaron realmente?
2. ¿Qué requisitos y decisiones confirmó el CLIENT?
3. ¿Qué estado técnico, owners y registros de control se materializaron?
4. ¿Qué verificaciones se ejecutaron físicamente y con qué resultado?
5. ¿Qué incertidumbre, HOLD o primera frontera de trabajo permitida queda?

Si el informe solo dice «se instalaron archivos» o «todo verde», la transición no quedó demostrada. Las herramientas sirven para registrar un estado verdadero; no permiten sustituir evidencia del producto por una etiqueta tranquilizadora.

6. El examen decisivo: contexto fresco

Abrí una sesión realmente nueva, sin entregarle la narrativa completa de la conversación anterior. Proporcioná únicamente la identidad del repositorio, el acceso de lectura autorizado y las instrucciones normales de entrada. Pedile que reconstruya su rol y límites, requisitos, ref de código, Product State, campaña y checkpoint si existen, gate abierto, mejor evidencia, hechos desconocidos, siguiente acción permitida y acciones prohibidas.

La respuesta debe fundarse en superficies durables. Repetir de memoria la arquitectura no demuestra continuidad. Tampoco pasa si confunde un documento sustituido con el actual, inventa estado de la DB, considera un PASS de I4 como aceptación o delega al CLIENT una contradicción técnica que debe resolver el owner del sistema.

Si el problema es falta de acceso, identificá y corregí ese permiso dentro del alcance autorizado. Si el propio estado canónico se contradice, reconciliá la causa y el mecanismo de currentness. No apruebes el examen simplemente por escribir un handoff de cien páginas para el próximo chat. Consultá el test de contexto fresco y su prompt de prueba.

7. Primera campaña autorizada

Elegí una capacidad que pueda construirse y verificarse de extremo a extremo. El charter expresa resultado, dependencias, ownership, interfaces afectadas, recursos que pueden interferir, gates pertinentes, evidencia exigible y condiciones de invalidación. No debería imponer veinte microcommits ni cuántos reintentos necesita I3 para corregir un bug.

No confundas «el control layer valida» con «la persona puede registrar un socio y conservar sus pagos». La primera frase habla de contratos del framework; la segunda necesita integración, persistencia, seguridad, funcionamiento físico y posiblemente una valoración humana de uso.

Según el riesgo, diferenciá gates de unidad, integración, DB/currentness, semántica/generalizació funcionamiento físico, experiencia humana y publicación. Ninguno reemplaza silenciosamente al otro. Una vez autorizado el charter, I3 asume implementación y rework dentro del alcance. Bugs, refactors, reintentos, migraciones previstas y hallazgos de review no justifican volver a estrategia. Una modificación real de objetivo, autoridad o interfaz crítica sí puede invalidar el encargo.

8. Ejemplo de sesión inicial con el Planner

Un pedido de entrada breve puede ser suficiente:

> Quiero desarrollar una aplicación para registrar socios y cuotas. Antes de planificar implementación, inspeccioná mi repositorio real y el starter ArquitecturaRepo de solo lectura. Guiame por las decisiones de producto pendientes. Separá lo confirmado de lo inferido; no inventes datos ni infraestructura. Prepará un baseline para que yo lo revise. Después de mi confirmación, elegí el perfil justificado, la estrategia técnica y un plan con hitos y fechas. No inicies una campaña material hasta satisfacer los gates de baseline y continuidad.

El Planner debería inspeccionar archivos y ramas antes de preguntarte qué archivo existe o qué SHA tiene una referencia que puede consultar. Sí puede preguntar cuántas personas operan el club, desde dónde, qué datos deben conservarse y cómo se distingue un pago vencido de uno cancelado. Cada pregunta debe afectar la solución, no extender innecesariamente la conversación.

9. Un baseline que el CLIENT pueda confirmar

«Hagamos un CRUD profesional» no es una especificación revisable. Un ejemplo más útil es:

Dimensión	Requisito propuesto para la primera versión	Estado
Usuario	Una persona administrativa usa la aplicación desde una computadora del club.	Pendiente de confirmación.
Datos	Socios, cuotas, pagos históricos e inscripciones.	Pendiente de confirmación.

Dimensión	Requisito propuesto para la primera versión	Estado
Regla de retención	Desactivar una ficha nunca borra sus pagos anteriores.	Pendiente de confirmación.
Funciones iniciales	Alta, búsqueda, edición y desactivación de socios.	Pendiente de confirmación.
Seguridad y red	Uso local; sin publicar el servicio en Internet inicialmente.	Pendiente de confirmación.
Interfaz	Formularios legibles, búsqueda útil y errores claros.	Pendiente de confirmación.
Recuperación	Definir intervalo de respaldo y pérdida máxima de registros aceptable.	Decisión abierta.
Exclusiones	Sin pagos online ni aplicación móvil en la primera versión.	Pendiente de confirmación.

Solo marcá un requisito como confirmado si el CLIENT realmente lo aprobó. Cuando el baseline refleje su decisión, persistí la versión y el momento de confirmación. Si más adelante se modifica, registrá qué sustituye y cuál es el impacto. No borres el historial de decisiones legítimas.

Luego el Planner registra tecnología, almacenamiento, red, backups y milestones con fechas, dependencias, evidencia de aceptación, confianza y riesgos. Las elecciones deben responder a la aplicación real; no a un stack de moda ni a la preferencia arbitraria del asistente.

10. Ejemplo de primera campaña

Una capacidad inicial razonable para el club sería: «Crear, consultar, modificar y desactivar socios sin perder historial de pagos, con errores comprensibles cuando los datos son inválidos». Puede demostrarse en una aplicación abierta y verificarse mediante persistencia real. El charter identifica límites del producto, reglas de datos, dependencias y gates. No necesita determinar cada archivo ni cada comando que ejecutará el I3.

I3 recibe ese objetivo autorizado y el código actual. Implementa, prueba, revisa, integra y corrige defectos en la misma campaña. Si el formulario rechaza un teléfono válido, lo arregla sin pedir otra estrategia. Si el CLIENT modifica el proyecto para atender cientos de clubes mediante un servicio público, cambia la necesidad y deben reconsiderarse requisitos, red, seguridad y alcance antes del reroute estratégico.

El resultado final debe tener un candidato y evidencia identificables. El Planner revisa gates técnicos y estratégicos; el CLIENT prueba el recorrido que le corresponde y puede aceptar o rechazar la experiencia. Una fixture verde no reemplaza una aplicación real ni la valoración de su usuario.

Siguiente: capítulo 03, Operación diaria.

Fuentes y vigencia

- Discovery, requisitos y plan de entrega
- Prompt de instalación y adaptación
- Prompt de discovery
- Prueba de contexto fresco
- Test de contexto fresco
- Contrato de transferencia y adopción
- Preflight de gobierno existente
- Work orders orientados al resultado
- Protocolo del Planner
- Código de baseline guard
- Código de work-order guard

Los ejemplos utilizan datos de club ficticios. No constituyen prueba de que un repositorio de destino haya sido inspeccionado, adoptado o aceptado.

03 — Operación diaria

ArquitecturaRepo · Manual completo de uso · Edición v1.4 · Español

> Este capítulo parte de un proyecto ya adoptado y con un charter vigente. El controlador opcional del starter ofrece mecanismos portables de referencia; no es una garantía de ejecución autónoma en Codex Desktop ni en otro host propietario. La compatibilidad de cada entorno debe comprobarse físicamente.

Contenido del capítulo

1. Preparar una sesión sin empezar de nuevo
2. Convertir un deseo en una capacidad comprobable
3. Qué hace I3 cuando el trabajo se complica
4. Autorizaciones y efectos externos
5. Cómo leer críticamente un reporte del ejecutor
6. Git y la continuidad
7. Cierre diario en cinco preguntas
8. Una jornada sin depender de la memoria del chat
9. Ejemplo de despacho delimitado
10. Interpretar estados de un controlador
11. Procesar el reporte final
12. Dejar una sesión recuperable

1. Preparar una sesión sin empezar de nuevo

Antes de pedir otra función, identificá el proyecto, el worktree, la rama y el ref actuales, los cambios locales pendientes, el Product State canónico, el charter activo y el último candidato físicamente comprobado. Si una sesión se cerró por un límite del servicio, el trabajador nuevo tiene que recuperar el checkpoint durable, no recrear «un proyecto parecido» desde un prompt viejo.

Cuando el checkout es confiable y el baseline está identificado, la sincronización comienza por examinar el delta exacto de Git y sus dependencias. No reclones ni ejecutes `git pull` automáticamente en cada reanudación si hay cambios de usuario sin integrar. Tampoco confíes ciegamente en un delta incremental cuando las ramas divergen o la autoridad de source/currentness está en disputa: primero se investiga el conflicto de solo lectura.

El CLIENT puede recibir un resumen compacto, sin tener que decidir qué SHA o snapshot de DB prevalece. Inspeccionar el repositorio verdadero, las instrucciones aplicables, los cambios locales y las referencias de ownership es responsabilidad técnica. El Planner reconstruye el objetivo y la campaña antes de autorizar una nueva capacidad. Si falta una evidencia indispensable, se declara el hecho desconocido y se detienen solamente las escrituras que dependen de él.

Podés colocar la planificación en una ventana y el editor en otra, pero esa comodidad visual no constituye un protocolo de seguridad. La continuidad surge del código, el estado y los checkpoints reconstruibles, no de mantener abiertas dos conversaciones.

Si alguien modificó código remotamente, el I3 debe reconciliarlo con el trabajo local antes de mutar.

2. Convertir un deseo en una capacidad comprobable

«Registrar pagos de cuotas, conservar el historial y mostrar quién está al día» es una capacidad observable. «Escribir la función de pagos» es una posible microtarea. El Planner comienza por el resultado que necesita el usuario, identifica interfaces, dependencias y riesgos, y autoriza el charter de esa capacidad. La implementación interna pertenece a I3.

Un encargo breve del CLIENT podría decir:

> Necesito que el personal registre pagos de cuotas sin perder movimientos anteriores y pueda ver quién debe. Por ahora no quiero pagos online. Priorizá una pantalla simple. Antes de aceptarlo quiero comprobar todo el recorrido.

La petición define intención, pero no necesariamente basta para iniciar ejecución: todavía hay que revisar requisitos actuales, reglas de datos, código, autoridad y entorno. El despacho del Planner contiene objetivo, fuentes verificadas, límites, interfaces, owners, gates y evidencia necesaria. No debería convertir al CLIENT en el coordinador de cada especialista.

En trabajo prolongado, costoso o por lotes, el work order orientado al resultado añade un inventario concreto, ruta operativa viable, presupuestos, condiciones de parada, salida y lectura física de los resultados. Complementa al charter sin reemplazarlo ni inventar autorizaciones de proveedores. Si la tarea consiste en comprobar archivos y hashes, una herramienta determinista puede resolverla sin solicitar una evaluación de un modelo pago por cada elemento.

3. Qué hace I3 cuando el trabajo se complica

Supongamos que el registro de un pago está implementado, pero falla una prueba de integración. I3 conserva la reproducción, identifica el componente causal, corrige el defecto, vuelve a ejecutar los tests pertinentes, integra el candidato y obtiene revisión independiente cuando corresponde. No necesita un nuevo charter por cada falla.

La depuración, los reintentos, los refactors internos, las migraciones previstas, la reparación de evaluadores, la integración y la recuperación tras una interrupción pertenecen continuamente al controlador I3 dentro del alcance autorizado. Si se repite la misma clase de problema, tiene que revisar el subsistema y sus tests de forma integral, no acumular remiendos locales o devolver el problema al Planner como una microcampaña.

Un bloqueo verdadero de currentness, la ausencia de autorización para una acción externa o un cambio material del objetivo se tratan de manera distinta. El I3 registra el límite exacto y al owner que puede resolverlo; no rebautiza el problema como «otro retry». A menudo puede seguir investigando de solo lectura mientras las escrituras dependientes permanecen en HOLD.

La revisión independiente debe cubrir el candidato exacto que se devuelve a auditoría. Si el código, la configuración o los artefactos relevantes cambian materialmente después de una revisión favorable, hay que renovar las comprobaciones afectadas. No se traslada automáticamente un PASS de la versión A a la versión B.

El `tools/campaignctl.py` portable ofrece mecanismos de referencia de control, pero no es un sistema universal de agentes distribuidos. Los helpers de Goal y plan táctico pueden administrar registros sin demostrar que un host externo activó comandos nativos como `/plan`. El runtime de referencia documenta lo que puede afirmarse de su implementación.

4. Autorizaciones y efectos externos

Acción	Tratamiento razonable
Leer código o ejecutar pruebas locales dentro del permiso otorgado.	Trabajo técnico rutinario autorizado.
Modificar e integrar código dentro de un charter aprobado.	Responsabilidad de I3 con ownership y worktree verificados.
Instalar dependencias o acceder a herramientas nuevas.	Comprobar procedencia, alcance, permisos y necesidad.
Desplegar, publicar, enviar correos, gastar crédito o borrar datos relevantes.	Autorización reservada específica y preflight proporcional.
Copiar un token o una base privada en el chat.	No hacerlo; usar el mecanismo aprobado de secretos y datos.
Encontrar contradicciones de código, DB, fuente o ownership canónicos.	HOLD para acciones dependientes y reconciliación basada en evidencia.

El CLIENT decide reglas, experiencia y riesgos de producto, además de publicación y gasto cuando corresponden. No necesita elegir el nombre de índices SQL, resolver conflictos de Git, diseñar transacciones o coordinar la recuperación. La arquitectura debe mantener esa frontera sin apropiarse de las decisiones que realmente le pertenecen.

5. Cómo leer críticamente un reporte del ejecutor

Un reporte útil identifica el candidato exacto y su entorno, el comportamiento que se intentó obtener, los cambios físicamente comprobados, las pruebas ejecutadas, los fallos y correcciones, el alcance de la revisión independiente y los gates todavía abiertos. No mide progreso por número de commits, tokens, agentes o horas de runtime.

Cuando aparece `PASS`, preguntá qué gate significa. Una prueba unitaria puede comprobar lógica con mocks; una de integración, la comunicación entre componentes; una de DB/currentness, persistencia y referencias; una física, que la aplicación realmente abra y permita completar una tarea; una perceptual, que la experiencia sea adecuada para una persona. Pasar una no demuestra necesariamente las otras.

El informe debe terminar indicando la próxima acción y su owner. Si falta una regla nueva del negocio, vuelve al CLIENT y al Planner. Si es un bug previsto por el charter, permanece en I3. Si hay un candidato listo, vuelve para auditoría estratégica consolidada. El CLIENT no debe convertirse en director técnico de incidentes.

Al aceptar una capacidad, el Planner registra qué candidato se aceptó, con qué charter, requisitos y evidencia. Al rechazar defectos dentro del alcance, devuelve un conjunto consolidado de falsificadores a la misma campaña. Un cambio genuino de producto se persiste como decisión nueva y no se disimula bajo el nombre de «bug fix».

6. Git y la continuidad

Git conserva código e historial, pero no sustituye Product State, registros de autorización ni estado de la base de datos. Un commit identifica contenido y padres; no prueba quién autorizó cada cambio, quién es el dueño de una campaña o si el usuario aceptó el producto. El movimiento de una rama tampoco demuestra currentness de datos reales.

No hagas un commit canónico por cada mensaje de I4. Los especialistas entregan cambios y evidencias acotados; la integración la realiza el owner correspondiente. Un worktree puede conservar trabajo del usuario sin commitear. No lo borres para obtener un status «limpio». Cuando trabajan varios actores, analizá la interferencia según los archivos, bases, runtime y recursos que comparten, no solamente según el nombre del proyecto.

Reutilizá un checkout confiable cuando sea seguro. Los worktrees temporales se justifican cuando existe concurrencia real, no como hábito. Las capturas, renders, exports y caches derivados deberían ser efímeros o ignorados salvo cuando constituyan evidencia canónica. El espacio en disco es un recurso que hay que proteger, sin destruir por ello registros actuales ni artefactos necesarios.

El mismo charter puede continuar tras el límite de uso de una sesión si el ref, checkpoint, owners y evidencia siguen reconstruibles. El límite de tokens de un servicio no invalida por sí mismo una campaña. Si un host ya no permite abrir el thread anterior, recurrí al estado canónico en vez de fingir que el historial sigue disponible.

7. Cierre diario en cinco preguntas

Antes de terminar una jornada, el sistema debería poder responder:

1. ¿Qué capacidad sigue activa y quién controla la ejecución?
2. ¿Qué ref de código, Product State y testigos de DB/currentness están realmente comprobados?
3. ¿Cuál fue el último gate que pasó, con qué evidencia?
4. ¿Qué defecto, riesgo, permiso o decisión de producto continúa abierto?
5. ¿Cuál es la próxima acción válida y quién la debe realizar, sin pedirle al CLIENT que arbitre cuestiones técnicas?

Persistí las decisiones materiales y próximas rutas en las superficies canónicas del repositorio. El chat puede conservar contexto conversacional, pero no debe ser la única fuente de verdad. El próximo trabajador tendría que recuperar las respuestas por su cuenta.

8. Una jornada sin depender de la memoria del chat

Al comenzar el día, recuperá el Goal vigente antes de pedir otra función. El Planner comprueba si hay campaña abierta, último ref integrado, evidencia pendiente y decisiones reservadas al CLIENT. Las rutas razonables son continuar I3, auditar un terminal consolidado, pedir una decisión de producto o autorizar una capacidad nueva después de satisfacer sus dependencias.

Si el trabajo se interrumpió por capacidad o por una caída, retomá desde un checkpoint real. Cuando no existe checkpoint, intentá reconstruir el estado durable.

Si tampoco hay evidencia suficiente, declarás qué se perdió y detenés las acciones dependientes. Una nueva sesión que solo afirma «íbamos por la mitad» no demostró que el proyecto esté a mitad de camino.

9. Ejemplo de despacho delimitado

Un mensaje de ejecución para una campaña ya autorizada puede ser compacto:

> **Goal:** implementar la gestión de socios prevista en el charter vigente.
>
> **Contexto:** leer el `AGENTS.md` real, requisitos confirmados, Product State, charter y código/ref actuales. Reconstruir ownership y currentness antes de mutar.
>
> **Restricciones:** conservar historial; respetar exposición local confirmada; no introducir pagos online ni funciones ajenas; no publicar ni gastar servicios externos.
>
> **Gates:** comprobar físicamente alta, búsqueda, edición, desactivación y persistencia al reiniciar; probar entradas inválidas y duplicados; revisar independientemente el candidato integrado.
>
> **Cierre:** entregar ref exacto, evidencia rastreable, resultado de revisión, riesgos y un terminal honesto. Resolver bugs, reintentos y refactors dentro de la misma campaña.

Este texto es un ejemplo, no una autorización universal para cualquier repositorio. Adaptalo al charter real. No inventes `AGENTS.md` ni Product State por aparecer en el ejemplo: hay que localizar las superficies actuales y declarar las ausencias concretas.

10. Interpretar estados de un controlador

Estado	Qué indica	Qué no demuestra
`READY`	Una unidad satisface las dependencias para poder ejecutarse.	Que ya se esté ejecutando o vaya a funcionar.
`ACTIVE`	Un contexto autorizado posee una ejecución activa.	Que el comportamiento final sea correcto.
`VERIFY`	Se preparan o realizan las comprobaciones pertinentes.	Que todos los gates del producto pasaron.
`HOLD` / `CAPACITY`	Hay una condición registrada que impide continuar.	Que haga falta abrir otra campaña.
`DONE` táctico	Una unidad de ejecución alcanzó sus condiciones locales.	Que Planner o CLIENT hayan aceptado la capacidad.
Terminal estratégico	I3 devuelve un candidato o bloqueo con evidencia.	Que exista autorización de publicación.

El runtime portable es una referencia, no una integración nativa universal. Si el

proyecto usa otro controlador, verificá que su implementación real preserve límites de autoridad, currentness, ownership, control de concurrencia y recuperación.

11. Procesar el reporte final

El Planner revisa candidato, requisitos actuales, entorno comprobado, resultados físicos, tests, review independiente y riesgos. No tiene que repetir el trabajo táctico de I3 ni aceptar un PASS por su etiqueta. Si la evidencia es incompleta, señala el gate específico sin declarar automáticamente que todo lo demás falló.

Para devolver un defecto dentro del charter, consolidá comportamiento observado, condiciones de reproducción, evidencia y gate bloqueado. I3 corrige e integra. Si vuelve reiteradamente la misma clase de falla, corresponde hardening del subsistema y revisión independiente, no una serie interminable de correcciones puntuales.

Una regla nueva de producto, permiso o interfaz incompatible puede justificar intervención estratégica. No se debe usar esa posibilidad para regresar a planificación cada vez que una prueba de regresión detecta un error conocido.

12. Dejar una sesión recuperable

Antes de cerrar, persistí el ref de código actual, objetivo vigente, checkpoint, evidencias de DB o artefactos cuando correspondan, mejores pruebas y review, bloqueos y ruta siguiente. Conservá datos privados fuera de documentación pública. Una release requiere su propio gate aunque una unidad táctica haya terminado.

El CLIENT debería poder preguntar «¿qué puedo usar hoy?» y recibir una respuesta sobre capacidades disponibles y riesgos importantes. La continuidad está demostrada cuando otra sesión recupera por sí misma esa respuesta desde el repositorio y el runtime autorizado, sin releer la conversación entera.

Siguiente: capítulo 04, Solicitar cambios.

Fuentes y vigencia

- Protocolo de ejecución
- Ciclo de vida y evidencia
- Runtime de referencia
- Eficiencia y cuidado del disco
- Work orders orientados al resultado
- Código Goal/Plan
- Código de host dispatch
- Controlador de campañas

Los mecanismos descritos son referencias y contratos. Su presencia en el repo no certifica el funcionamiento de ningún host externo ni aplicación específica.

04 — Solicitar cambios

ArquitecturaRepo · Manual completo de uso · Edición v1.4 · Español

> Las decisiones de producto pertenecen al CLIENT. El Planner las traduce a requisitos confirmados, límites técnicos, hitos y gates verificables. I3 puede modificar su plan táctico dentro de un charter autorizado, pero no reescribir por su cuenta el Goal estratégico, los efectos prohibidos ni la definición del producto.

Contenido del capítulo

1. Empezar por el resultado, no por el código
2. Una ficha breve para formular la necesidad
3. Cuáles son tus preguntas y cuáles no
4. Cambiar de opinión sin destruir el historial
5. Pedir calidad sin imponer una tecnología
6. Priorizar en vez de abrir diez Goals
7. Cuando el Planner necesita una decisión real
8. Cinco requisitos bien formulados
9. Cambio de producto frente a defecto de implementación
10. Qué decir cuando «funciona» pero no sirve
11. Guion corto para comunicar una prioridad

1. Empezar por el resultado, no por el código

El CLIENT describe quién usa una función, qué quiere hacer, cuál es el resultado observable, qué condiciones deberían impedirla y qué información debe conservarse. El Planner convierte esa intención en requisitos confirmados, dependencias, límites de implementación y condiciones de aceptación. Podés tener preferencias técnicas, pero las decisiones rutinarias de APIs, tablas, tests y estructura de código deben resolverlas los responsables de ingeniería.

«Agregá un botón para cancelar» puede ser insuficiente. Si la cancelación modifica cupos, pagos, permisos o historial, dibujar el botón no alcanza. La pregunta útil es: «Cuando se cancela una inscripción, ¿el lugar vuelve a quedar disponible? ¿Qué queda guardado en el historial?». La respuesta determina qué comportamiento hay que implementar y demostrar.

Separar problema y solución permite explorar alternativas. Un Planner puede proponer un botón, una acción contextual o una pantalla distinta; lo fundamental es que respeten la regla de cancelación confirmada. Una maqueta visual ayuda a conversar sobre interfaz, pero no demuestra que el estado de la aplicación ni los errores estén resueltos.

2. Una ficha breve para formular la necesidad

No hace falta escribir una especificación extensa para cada cambio. Cuando importa la conducta del producto, describí actor, situación inicial, acción, resultado visible,

exclusiones y datos que sobreviven. Por ejemplo:

> Como administrativa del club, quiero desactivar un socio sin borrar sus pagos anteriores. No debe aparecer en la búsqueda de activos, pero una persona autorizada debe poder consultar sus registros históricos.

El Planner todavía tiene que aclarar si una ficha puede reactivarse, qué identificadores son únicos y qué datos están sujetos a retención. El CLIENT no necesita diseñar el esquema SQL ni escribir tests para pedir ese comportamiento.

Una ficha concreta de inscripción podría ser:

Campo	Ejemplo
Persona usuaria	Personal administrativo del club.
Situación	Llega un socio que quiere participar en una actividad.
Resultado	La inscripción se guarda solo si hay cupo y se cumplen las reglas actuales.
Exclusiones	Sin pagos online ni avisos automáticos en esta etapa.
Información que permanece	Identidad, actividad, fecha, estado y modificaciones posteriores.
Aceptación	Puedo inscribir, ver cupos restantes y cancelar sin perder historial.

No conviertas la ficha en una burocracia de diez campos para arreglar un detalle menor. Su función es eliminar ambigüedad material. Las decisiones de producto que cambian requisitos vigentes deben persistirse; un mensaje en un chat no basta como estado recuperable.

3. Cuáles son tus preguntas y cuáles no

Una pregunta es del CLIENT si modifica el comportamiento o el riesgo de negocio: si una deuda bloquea inscripciones, si una ficha puede recuperarse, si el programa debe funcionar sin Internet, si se autoriza gastar, publicar o enviar mensajes. Son decisiones sobre personas y uso previsto.

La estructura de tablas, el framework de pruebas, las ramas, los logs, los algoritmos de reintento y las migraciones pertenecen a ingeniería. El Planner puede explicar consecuencias cuando una elección técnica afecta costo, privacidad, compatibilidad o la experiencia, pero no debería pedirle al CLIENT que resuelva un error de SQL porque el ejecutor no pudo depurarlo.

Cuando hay opciones legítimas de producto, presentalas con efectos comprensibles: «Con lista de espera, el personal tiene que confirmar el ingreso cuando se libera un lugar; sin lista de espera, el sistema rechaza la inscripción inmediatamente». Indicá el impacto y recomendá si hay evidencia suficiente. No es necesario dar una clase de arquitectura para obtener una respuesta.

4. Cambiar de opinión sin destruir el historial

Los requisitos pueden cambiar. Hay que registrar **cuál** decisión cambia, **por cuál** se sustituye y **desde cuándo**. Por ejemplo:

> Antes no se permitía inscribir socios con cuotas vencidas. Ahora se permite, pero la deuda debe aparecer claramente en la vista administrativa. Registrá la regla anterior como sustituida y evaluá el impacto en código, datos, pantallas y pruebas.

La nueva regla puede afectar validaciones, persistencia, UI, pruebas y campañas activas. El Planner rastrea esas relaciones y actualiza el requisito canónico vigente. Que una decisión anterior haya sido válida no significa que deba seguir activa para siempre. Lo incorrecto sería conservar dos archivos `CURRENT` que exijan comportamientos opuestos sin una supersesión explícita.

Otro cambio posible:

> Ya no quiero eliminar definitivamente fichas de socios desde la interfaz. Quiero desactivarlas y conservar pagos e inscripciones. Tratá la política anterior como reemplazada, comprobá qué sucede con datos existentes y explicame el nuevo recorrido de usuario.

El equipo técnico diseña migración, recuperación, tests e interfaces. El CLIENT confirma el nuevo comportamiento y los permisos reservados. La siguiente sesión tiene que encontrar la decisión actual en las superficies del repositorio, sin depender de que alguien recuerde el último mensaje.

Si el pedido transforma completamente el producto —por ejemplo, de una aplicación interna de un club a un servicio público para cientos de clubes— puede requerirse invalidación o rediseño del charter estratégico. Si simplemente se detectó que un campo no valida como exige el requisito vigente, I3 corrige en la misma campaña.

5. Pedir calidad sin imponer una tecnología

«Que quede profesional» orienta, pero no basta como criterio de aceptación. Contá qué sería incorrecto o molesto para una persona: no encontrar una ficha, perder datos al corregir un error, no saber si el pago quedó guardado, recibir mensajes incomprensibles o activar una acción irreversible sin confirmación.

Un pedido útil es:

> La pantalla de cuotas debe mostrar estado, importe y fecha del último pago sin abrir varias ventanas. Si el guardado falla, no debe parecer exitoso. Prefiero claridad y consistencia antes que una decoración sofisticada.

Esto permite plantear pruebas de integración y condiciones de uso sin elegir un framework visual. Una captura real con una crítica específica suele ser más útil que diez adjetivos. Cuando la calidad depende de la experiencia, la evaluación del CLIENT puede ser un gate perceptual genuino; una prueba unitaria o una puntuación automática no deberían suplantar esa decisión.

También importa el comportamiento negativo. Una pantalla puede guardar un pago en el caso ideal y duplicarlo después de dos clics; puede aceptar datos válidos pero perder el resto del formulario al mostrar un error. El Planner vincula los riesgos con gates pertinentes; I3 define cómo implementar y comprobar las correcciones.

6. Priorizar en vez de abrir diez Goals

La prioridad señala cuál es la próxima capacidad que debe volverse utilizable respetando dependencias y recursos. En el club, una secuencia razonable sería:

socios confiables, luego cuotas y después actividades e inscripciones. Si el tercer paso necesita datos del primero y del segundo, el Planner registra la relación. No hay beneficio en autorizar diez capacidades a la vez solo porque un asistente pueda generar diez planes.

Frase vaga	Pedido útil
«Hacé todo el sistema del club».	«Primero dejá verificable la gestión de socios; después integrá cuotas».
«Arreglá lo que haya».	«Al cancelar una inscripción no se libera el cupo; corregilo sin borrar historial y demostrámelo».
«Hacelo más moderno».	«Reducí los pasos para encontrar un socio y revisar su deuda; priorizá legibilidad».
«Agregá todo lo que alguien podría necesitar».	«Validemos primero el recorrido administrativo principal antes de ampliar alcance».

Si la prioridad cambia durante la ejecución, el sistema debería cerrar correctamente la frontera abierta o dejar un checkpoint recuperable. Un cambio repentino no autoriza a borrar trabajo sin integrar, reemplazar silenciosamente un Goal ni perder el estado de campaña. El Planner considera dependencias, interferencia y camino crítico.

7. Cuando el Planner necesita una decisión real

La pregunta debería describir el problema en lenguaje de uso, alternativas viables, efectos sobre costo o riesgo y cuál es exactamente la elección reservada al CLIENT. «¿El personal puede reactivar una inscripción cancelada?» es útil. «¿Qué índice SQL agrego para la migración?» normalmente no lo es.

Ante una consulta de estado, pedí tres datos: qué comportamiento existe y se comprobó; qué falta para el objetivo activo; y si hay alguna decisión de producto que solo vos podés resolver. Una respuesta que enumera commits, horas, tokens y nombres de agentes sin explicar qué puede hacer el usuario no es una evaluación honesta del avance.

Si aún no tenés una respuesta para una decisión material, señalala como abierta. El equipo puede continuar el trabajo seguro que no dependa de ella. La ingeniería no debe convertir una preferencia no confirmada en un requisito autorizado.

8. Cinco requisitos bien formulados

Búsqueda

> Quiero encontrar socios por nombre o número y distinguir activos de desactivados. Si no hay coincidencias, mostrámelo con claridad. No quiero exponer datos personales a usuarios no autorizados.

Define comportamiento, estado vacío y privacidad sin imponer un motor de búsqueda. Permite comprobar consultas autorizadas y denegadas.

Edición

> Necesito corregir el teléfono de un socio. Antes de guardar debe quedar claro qué ficha estoy editando. Si ingreso un dato inválido, conservá los otros campos y explicá el error.

Describe navegación, retención del formulario y feedback; probar solo el endpoint no demuestra la interacción completa.

Desactivación

> Desactivar una ficha la excluye de la lista de activos, conserva los pagos y permite que personal autorizado consulte el historial.

La palabra desactivar tiene un significado comprobable. Una exigencia posterior de eliminación legal se trataría como una regla distinta.

Rendimiento

> La búsqueda debe seguir siendo utilizable con la cantidad de socios que esperamos durante el primer año. Si necesitás una estimación para dimensionarla, preguntámela.

Vincula rendimiento con uso real sin imponer prematuramente caching o una base de datos.

Nueva capacidad

> Agregá inscripciones después de que socios y cuotas sean utilizables. Primero acordemos cupos, cancelaciones, vencimientos y permisos; después planificá la integración.

Evita construir un componente que depende de capacidades incompletas o reglas contradictorias.

9. Cambio de producto frente a defecto de implementación

Si el requisito vigente prohíbe inscripciones a socios morosos y el programa las permite, la implementación incumple una regla: I3 la corrige dentro del alcance autorizado. Si nunca se decidió cómo tratar pagos parciales, el Planner consulta antes de inventar la conducta. Si ahora se quiere que un familiar pueda pagar la cuota de otra persona, aparece una regla nueva que puede afectar identidad, permisos e historial.

La diferencia es importante: ingeniería no debería exigir autorización por cada bug, pero tampoco apropiarse de una preferencia nueva. Las decisiones materiales cambian requisitos actuales con trazabilidad, de forma que el próximo agente pueda recuperar la regla válida.

Una revisión del plan táctico no equivale automáticamente a un cambio de producto. El Goal estratégico puede tener `goal_id`, `campaign_id`, `charter_ref`, `authority_refs`, `definition_of_done`, `hard_gates`, `forbidden_effects` y `write_envelope`. I3 puede

corregir pasos, hipótesis y tests dentro de ese contrato. No puede convertir unilateralmente un gasto o efecto prohibido en algo autorizado.

10. Qué decir cuando «funciona» pero no sirve

Podés rechazar una capacidad por un recorrido de usuario inutilizable aunque el guardado técnico funcione. Por ejemplo:

> El formulario guarda los datos, pero en el celular el botón de confirmar queda fuera de pantalla. No acepto el recorrido hasta que pueda completarse desde ese dispositivo.

Una captura o una descripción precisa del dispositivo, estado y resultado esperado sirve como evidencia perceptual. Si el uso móvil estaba expresamente excluido del alcance, incorporarlo puede ser un cambio de producto, no un bug ordinario. El Planner debe aclarar esa frontera y revisar fechas cuando corresponda. Si la compatibilidad móvil ya era un requisito, I3 repara y renueva la evidencia pertinente dentro de la misma campaña.

La valoración positiva del CLIENT tampoco reemplaza los tests técnicos. Una interfaz agradable puede conservar mal los datos o fallar en permisos. Una capacidad aceptable satisface los gates que su riesgo exige, cada uno con su evidencia y autoridad.

11. Guion corto para comunicar una prioridad

Podés decirle al Planner:

> Mi prioridad actual es [resultado visible]. La persona usuaria es [quién]. Debe poder [recorrido comprobable] y no debe [restricción principal]. La fecha objetivo es [fecha] y el costo máximo es [presupuesto, si existe]. Antes de programar, decime qué decisiones de producto siguen abiertas y qué capacidades vigentes dependen de este cambio. La tecnología, las pruebas, la integración y la recuperación dentro del alcance quedan a cargo de ingeniería.

No es necesario copiar el texto literalmente. Lo importante es que tu intención quede reconstruible y verificable sin transformar cada sugerencia casual en un compromiso irreversible. Un buen Planner sabe cuándo preguntar, cuándo leer evidencia física y cuándo dejar ejecutar a I3.

Siguiente: capítulo 05 para errores y permisos sensibles; capítulo 06 para aceptación de capacidades.

Fuentes y vigencia

- Fronteras de decisión del CLIENT
- Registro de interacciones y decisiones
- Prompt de discovery
- Requisitos y baseline técnico
- Ciclo de vida y evidencia

Los ejemplos son requisitos ilustrativos. No constituyen una decisión real del CLIENT sobre una aplicación concreta.

05 — Errores, incidentes y recuperación

> El objetivo de recuperar un proyecto es volver a un estado **verdadero y verificable**, no simplemente hacer desaparecer un mensaje de error. Se conservan fuente, ownership, journal de transacciones, testigos de DB y evidencia vigente. Un bug ordinario dentro de un charter pertenece a I3; la falta de autorización o un conflicto de currentness canónica exige otra ruta.

Contenido del capítulo

1. Primero conservar evidencia
2. Cuando falla el entorno o una cuenta
3. Si no valida el control layer
4. Git no es un botón para «deshacer problemas»
5. Un fallo repetido exige cambiar la causa
6. Seguridad, privacidad y derechos de terceros
7. Salir de un incidente con pruebas
8. Tres incidentes completos y su recuperación
9. Matriz para operaciones sensibles
10. Qué no demuestra una validación verde
11. Privacidad y feedback del starter

1. Primero conservar evidencia

Guardá el mensaje exacto, el comando o acción que lo produjo, el momento, el repositorio y worktree afectados, y el ref de código o artefacto cuando exista. No borres la carpeta, reinstales todo, descartes cambios locales ni crees otro clon por reflejo. Esas medidas pueden destruir la información que el responsable técnico necesita para recuperar el proyecto.

Antes de reparar, hay que clasificar el problema: herramienta faltante, permiso, defecto de aplicación, contrato de gobierno inválido, fuente desactualizada, transacción interrumpida, proveedor externo o autorización reservada. Son incidentes distintos y sus soluciones no son intercambiables.

Señal	Respuesta inicial segura
Falla una prueba de integración de la feature activa.	I3 conserva reproducción y corrige en el charter vigente.
El validador rechaza un registro canónico.	HOLD de acciones dependientes e identificación de la relación y owner causales.
Hay cambios Git locales y remotos incompatibles.	Preservar ambas versiones, inspeccionar delta y conciliar sin adivinar.

Señal	Respuesta inicial segura
Los datos reales de usuarios podrían estar dañados.	Detener escrituras sensibles, conservar backups, WAL y journal, evaluar restauración.
Falta permiso para publicar o gastar.	No ejecutar el efecto externo hasta contar con autorización específica.
El host pudo despachar una operación antes de caer.	Registrar el efecto como desconocido y realizar readback antes de reintentar.

No se trata de que haya menos indicadores rojos, sino de corregir la causa y demostrar que el gate correspondiente pasa en el candidato actual. A menudo se puede continuar con análisis de solo lectura mientras las mutaciones materiales permanecen bloqueadas.

2. Cuando falla el entorno o una cuenta

Python o un comando no se reconoce. Verificá ruta, ejecutable, versión y entorno activo. Instalá las dependencias declaradas desde fuentes confiables. No modifiques Product State para compensar una herramienta ausente. Si el agente de programación puede inspeccionar la máquina, que examine el entorno; si no, conservá la salida completa para quien tenga acceso.

GitHub no aparece en ChatGPT. Distinguí autenticación, instalación del conector, permisos, alcance de repositorios y visibilidad de la rama. Un error de lectura no justifica dar acceso a toda la cuenta, entregar credenciales o cambiar archivos. Probá una lectura del repositorio concreto dentro del permiso autorizado.

El ejecutor abre otro proyecto. Confirmá que la carpeta existe y corresponde a la aplicación, no al starter genérico `ArquitecturaRepo/`. Antes de escribir, comprobá worktree, rama, archivos y permisos. Dos directorios parecidos no son intercambiables.

Se interrumpe la sesión o se alcanza el límite de uso. La sesión comercial no define la identidad de la campaña. Recuperá el Goal, owner, checkpoint, candidato integrado y efectos pendientes desde evidencia durable. El límite de una conversación no autoriza abrir otra campaña ni un controlador competidor. Si el servicio no puede recuperar el thread anterior, el repositorio sigue siendo la fuente de continuidad.

3. Si no valida el control layer

El `tools/validate_project.py` instalado examina contratos y referencias de `.agentic-sdlc/`. Puede rechazar una autoridad inexistente, un owner duplicado, una relación candidato/review incompatible o una referencia `CURRENT` mal formada. No demuestra por sí solo que la aplicación se conecte a su DB, que alguien pueda iniciar sesión o que los pagos sobrevivan a un reinicio.

Antes de tocar estado, conservá el error, el root del control, ref actual de Git, cambios locales y testigos relevantes de DB. El owner técnico determina si falta una dependencia, cambió el contrato, hay una referencia stale, una transacción parcial o una modificación externa conflictiva.

No escribas `READY` en un JSON ni borres el journal para obtener una pantalla verde. Si dos actores reclaman el mismo efecto o un candidato cambió después de una revisión

favorable, la solución es reconstruir autoridad y lineage. Un ajuste cosmético puede hacer que el informe parezca correcto mientras empeora el estado real.

Las transiciones pueden actualizar varios archivos canónicos. El framework describe recuperación transaccional basada en hashes antes y después. Un journal pendiente puede bloquear legítimamente otras escrituras. Si un tercero modificó uno de los destinos, la recuperación debe registrar `CONFLICT`, no sobrescribirlo por comodidad. Solo se reanuda cuando las relaciones pertinentes concuerdan y el resultado de recuperación fue comprobado físicamente.

En SQLite, el archivo principal `.db` no siempre representa todo el estado vivo si se utiliza Write-Ahead Logging. El WAL y un snapshot consistente pueden ser indispensables. Un hash del archivo principal aislado no autentica automáticamente el contenido actual. La ruta de attachment de gobierno existente contempla esta distinción, pero no repara automáticamente una base de producción.

4. Git no es un botón para «deshacer problemas»

Cuando GitHub Desktop o la consola muestran un conflicto, no elijas ciegamente «mi versión» o «la remota». Las dos pueden contener trabajo valioso. Primero inspeccioná la base común, los cambios locales, el delta remoto y el ownership vigente. No borres un checkout confiable simplemente porque el diff resulte incómodo.

El starter incluye `tools/repo_delta.py` para inspección incremental acotada y `tools/workspace_gc.py` para limpieza controlada. Ninguna herramienta habilita a borrar código fuente, datos, artefactos aceptados, archivos humanos desconocidos o evidencia canónica. Empezá con inventario read-only y dry-run. El borrado requiere demostrar que el recurso pertenece al proceso correspondiente y es efectivamente descartable.

Mantené un checkout persistente por repositorio cuando sea posible. Los worktrees temporales se justifican por concurrencia real; los clones adicionales, caches grandes y copias duplicadas de evidencia no deben acumularse «por si acaso». A la vez, la higiene del disco no es excusa para destruir lineage, backups, registros de transacciones activas o artefactos que pueden ser necesarios para falsificar una capacidad.

Comandos destructivos como `git reset --hard`, reemplazos forzados de rama o borrados recursivos no son recetas generales de recuperación. Si realmente se necesita rollback, identificá objetivo, punto de restauración, datos afectados y autorización antes de actuar. Retroceder un commit Git no revierte automáticamente transacciones de la DB.

5. Un fallo repetido exige cambiar la causa

Repetir una orden con los mismos inputs, entorno e hipótesis casi nunca reduce el riesgo. Cuando reaparece la misma clase de fallo, I3 consolida observaciones, examina el subsistema y sus interfaces, fortalece las pruebas relevantes y solicita revisión independiente. Sigue siendo la misma campaña salvo una invalidación estratégica real.

El work order acotado define presupuestos, límites de reintento y condiciones de salida cuando hay costo relevante. Si varios intentos no entregan resultados, I3 cambia materialmente el método o registra el bloqueo causal preciso. El Planner no debería emitir una secuencia interminable de nuevas campañas para renombrar el

mismo defecto. Tampoco se acepta el resultado solo porque una prueba unitaria pasó tras el último parche cuando el flujo físico continúa roto.

Un terminal útil puede consistir en una capacidad físicamente entregada con verificaciones pertinentes, un bloqueo preciso con evidencia y owner, o un checkpoint recuperable y una acción siguiente válida. Contar commits, horas o llamadas a modelos no reemplaza esos resultados.

6. Seguridad, privacidad y derechos de terceros

Un charter técnico no concede permiso implícito para exponer datos privados, consumir una API paga, divulgar credenciales, borrar registros protegidos o publicar contenido ajeno. Esas operaciones tienen límites de autoridad propios. Aplicá permisos mínimos, datos sintéticos en demostraciones no autorizadas, secretos gestionados por el entorno, tratamiento lícito de datos y verificación de licencias según el uso real.

- No coloques contraseñas, tokens, claves privadas ni cadenas de conexión en prompts, issues, historial Git o logs públicos.
- Usá datos ficticios o debidamente desidentificados en reviews y demos que no requieran datos personales reales.
- Antes de una migración o limpieza destructiva, determiná registros afectados y comprobá backup y restauración apropiados.
- Antes de publicar imágenes, código, música o video de terceros, verificá procedencia, licencia, atribución y alcance del uso. La licencia MIT del starter no cubre recursos ajenos.
- Antes de usar proveedores pagos, obtené un presupuesto autorizado y límites de reintento y costo. Un error de conexión no justifica llamadas ilimitadas.
- Antes de exponer la app a Internet, comprobá autenticación, autorización, configuración de red, datos, secretos y autorización de despliegue.

Privacidad, propiedad intelectual, seguridad y funcionamiento son gates relacionados pero distintos. Que una persona acepte el aspecto visual de un recurso no crea derechos para publicarlo. Que un test técnico pase no autoriza publicar datos personales. Conservá cada decisión con su owner competente.

7. Salir de un incidente con pruebas

El informe de recuperación explica qué falló, por qué, qué cambió, qué código y datos se conservaron o restauraron, qué pruebas físicas se ejecutaron y qué riesgos siguen abiertos. Identifica el candidato actual y el dueño de la próxima acción. I3 comprueba la reparación técnica; el Planner evalúa si la capacidad recuperó sus condiciones estratégicas.

Si el estado actual no puede reconstruirse o no existe un camino confiable para recuperar datos críticos, hay que decirlo. Un HOLD específico con un testigo faltante es preferible a fingir que el incidente terminó. Se puede continuar la investigación segura sin autorizar mutaciones sobre un estado incierto.

8. Tres incidentes completos y su recuperación

Incidente A — La sesión del agente termina durante una campaña

El host se desconecta mientras se implementa búsqueda de socios. No envíes la misma petición abstracta a otra sesión aislada esperando que el trabajo sea equivalente. Inspeccioná el checkout real y sus cambios sin integrar, journal de campaña, último checkpoint, ownership y lease, recibos de efectos pendientes y candidato integrado.

Si la fuente y la autoridad concuerdan, I3 reanuda la misma campaña desde el siguiente paso permitido. Si falta una referencia crítica, registra el testigo exacto que se debe recuperar y mantiene HOLD para lo dependiente. Un límite de uso no agrega requisitos ni invalida automáticamente el charter.

Incidente B — Se interrumpe una actualización de estado de varios archivos

Una transición iba a actualizar bootstrap, Product State, ownership y checkpoint, pero se corta a la mitad. Varios archivos podrían presentar versiones actuales incompatibles. El journal transaccional permite distinguir cambios aplicados, pendientes y conflictivos. Hay que inspeccionarlo antes de continuar escribiendo.

Utilizá la recuperación documentada de la herramienta responsable de la transición, por ejemplo `campaignctl recover` cuando corresponda al runtime de referencia. El mecanismo avanza solo si los hashes before/after justifican cada operación. Si otra persona editó el mismo archivo, el resultado puede ser `CONFLICT`. No borres el journal ni edites estados a mano para quitar el HOLD.

Incidente C — Los tests pasan, pero la aplicación real falla

El cálculo de cuotas pasa pruebas unitarias, pero la pantalla muestra un importe anterior después de guardar. Registrá candidato, pasos, entorno, salida esperada y observada, y estado de DB. El gate unitario pasó; el gate end-to-end o de persistencia no. Ejecutá el recorrido afectado y reparará el enlace entre UI, backend o estado compartido.

Si está en el charter vigente, I3 realiza la corrección, repite las pruebas físicas y de datos, integra un candidato nuevo y renueva la revisión pertinente. Repetir el mismo test unitario no prueba que se solucionó una sincronización de interfaz. La recuperación termina cuando el recorrido y las comprobaciones adyacentes afectadas realmente funcionan.

9. Matriz para operaciones sensibles

Acción propuesta	Riesgo principal	Límite previo
Mostrar una demo con fichas reales.	Exposición de datos personales.	Preferir datos ficticios; si es necesario, autorización, finalidad y protección adecuadas.
Crear un token de GitHub.	Acceso excesivo o filtración.	Permiso mínimo, secreto fuera del repo y revocación si se expone.

Acción propuesta	Riesgo principal	Límite previo
Borrar carpetas supuestamente temporales.	Pérdida de trabajo o evidencia.	Identidad del owner, descartabilidad y dry-run comprobados.
Publicar la aplicación en Internet.	Ataques, privacidad y gastos.	Autorización del CLIENT, preflight de seguridad/red y recuperación.
Activar una API paga.	Costos o reintentos sin límite.	Presupuesto, topes, medición y salida de error definida.
Publicar medios de terceros.	Uso sin derechos suficientes.	Fuente identificada, licencia y condiciones aplicables.

Un «sí, podés trabajar en mi carpeta» no equivale a permiso para efectos irreversibles, externos o pagos. La ingeniería implementa controles; las decisiones reservadas de producto y exposición deben quedar explícitas.

10. Qué no demuestra una validación verde

`validate_project.py` puede comprobar coherencia estructural del control layer. No prueba que el backend responda, que la DB sobreviva a un reinicio, que los permisos de los endpoints sean correctos o que una persona pueda completar la tarea. Un test sintético de adopción no autoriza migrar una DB real sin comprobar ese caso. Una captura bonita tampoco reemplaza la interacción física.

Después de reparar un incidente, repetí el gate que falló y las comprobaciones cercanas que la corrección pudo afectar. Identificá el candidato y quién revisó su estado actual. No invalides toda la evidencia estable sin razón, pero tampoco promociones cambios materiales con reviews viejos. Cuando código, DB y Product State se contradicen, mantené HOLD para los efectos dependientes.

La distribución también tiene una frontera: un source build verde no prueba por sí solo el contenido, digest o disponibilidad de un ZIP ya publicado. El readback de la release comprueba el artefacto exacto y constituye un gate distinto.

11. Privacidad y feedback del starter

En v1.4, la configuración local del reportador de metadatos estrictos está habilitada por defecto. El instalador puede crear un workflow periódico en GitHub Actions. El paquete público no incluye un collector externo; el envío requiere los opt-ins explícitos documentados. Revisá la política de feedback y cualquier workflow ejecutable antes de autorizar efectos de red.

`--no-feedback-workflow` evita instalar el programador, pero **no** apaga el generador local. Para desactivar toda generación, configurá `enabled=false` en el archivo de feedback instalado. No describas una opción como «sin telemetría» sin comprobar qué parte del mecanismo realmente desactiva.

Estas políticas del starter tampoco certifican que una aplicación desarrollada después trate correctamente sus propios datos personales. Cada producto debe revisar sus secretos, logs, flujos de datos, permisos, red y retención reales.

Siguiente: capítulo 06, Cierre y aceptación de capacidades.

Fuentes y vigencia

- Seguridad, privacidad, derechos y acciones externas
- Recuperación transaccional
- Feedback con privacidad
- Preflight de gobierno existente
- Implementación de transacciones
- Limpieza de workspaces
- Inspección incremental del repo

Estos incidentes son hipotéticos. Una recuperación real necesita evidencia directa del entorno afectado y autorización proporcional.

06 — Cierre de capacidades y aceptación

> Una capacidad se cierra contra un **candidato exacto y evidencia aplicable**, no contra el entusiasmo de un reporte. Un terminal de I3 habilita la revisión estratégica; no aprueba automáticamente el producto, no sustituye una revisión humana y no concede permiso para desplegar o publicar. Este capítulo separa esas decisiones y explica cómo devolver defectos a la misma campaña sin reiniciar la arquitectura.

Contenido del capítulo

1. Una capacidad es un resultado de producto
2. Los cinco componentes del cierre
3. El candidato exacto como unidad de revisión
4. Evidencia insuficiente sin campañas artificiales
5. Aceptar, recuperar o cambiar el objetivo
6. Revisiones periódicas y regla de detención
7. Registro de cierre reconstruible
8. Ejemplo de rechazo y corrección
9. Producto utilizable frente a framework verde
10. Qué hacer después del cierre

1. Una capacidad es un resultado de producto

«Gestión de socios» no está terminada cuando aparece un formulario bonito ni cuando un agente hace un commit. Está terminada cuando la persona prevista puede realizar la tarea acordada en el entorno pertinente, los datos relevantes permanecen como corresponde, los controles aplicables pasan, el estado vigente y su lineage son coherentes y las autoridades competentes aceptaron el resultado.

El Planner delimita la capacidad y autoriza un charter de ingeniería. I3 integra un candidato concreto, realiza las pruebas, organiza la revisión independiente cuando corresponda y presenta un terminal respaldado por evidencia. El Planner audita ese terminal contra los requisitos vigentes y la aceptación pactada. El CLIENT conserva las decisiones de producto, preferencias perceptuales y autorización de efectos externos que le correspondan.

Una campaña no dura obligatoriamente una conversación, un sprint, un número fijo de prompts ni una sesión de Codex. Puede sobrevivir desconexiones, límites de servicio, reintentos y checkpoints. Esos hechos no crean una autorización nueva ni permiten olvidar pruebas anteriores; tampoco convierten la campaña en un éxito por el solo hecho de haber trabajado muchas horas.

La distinción es especialmente importante para productos con interfaz. Un screenshot muestra un estado visual; no demuestra por sí mismo que el botón guarde datos, que un usuario tenga los permisos correctos o que pueda recuperar lo guardado después de reiniciar. La aceptación del producto necesita observaciones relacionadas con el comportamiento real.

2. Los cinco componentes del cierre

Componente	Pregunta que debe responderse
Resultado visible	¿La persona prevista puede completar la tarea en el entorno relevante?
Verificaciones aplicables	¿Se ejecutaron los controles de lógica, integración, DB, seguridad, runtime y experiencia que efectivamente corresponden?
Estado actual coherente	¿Se conoce la identidad exacta de código, configuración, requisitos y datos o artefactos afectados?
Sin bloqueos materiales ocultos	¿Existe algún defecto abierto que invalide lo que se afirma haber entregado?
Aceptación competente	¿Revisaron las autoridades pertinentes evidencia real, en vez de aceptar un `DONE` genérico?

No todas las capacidades requieren todos los ensayos posibles. Un pequeño script interno puede no necesitar una evaluación perceptual compleja. Un sistema que registra cuotas sí tiene que demostrar persistencia y permitir seguir correctamente el recorrido de la persona que lo opera. Lo que no puede pasar es que un gate reemplace silenciosamente a otro: validar la forma de un JSON no equivale a probar el CRUD; una prueba unitaria no demuestra que una pantalla cargue en el navegador.

En ArquitecturaRepo conviene distinguir los siguientes gates:

- **Unit:** la lógica aislada se comporta correctamente con entradas controladas y casos negativos.
- **Integration:** los componentes que se comunican cumplen interfaces y contratos reales.
- **DB/currentness:** registros, migraciones, relaciones, snapshots y testigos del estado actual son coherentes y suficientemente durables.
- **Semantic/generalization:** el comportamiento responde a la regla de negocio, incluso más allá de una fixture estrecha.
- **Physical:** el ejecutable, aplicación, artefacto o recorrido funciona realmente donde se pretende utilizar.
- **Perceptual/human:** una persona calificada evalúa experiencia, legibilidad o criterios subjetivos aplicables.
- **Release:** el paquete exacto, permisos, derechos, firmas o checksums y condiciones de distribución están verificados.

La cantidad y profundidad de pruebas debe ser proporcional al impacto. Pedir docenas de controles sin capacidad de descubrir defectos materiales puede demorar el producto; omitir los que importan genera aceptación falsa. Un gate útil tiene owner, criterio, candidato y evidencia. Si no se puede explicar qué riesgo reduce, vale la pena revisar si corresponde.

3. El candidato exacto como unidad de revisión

El revisor independiente tiene que inspeccionar el mismo candidato que luego se propone aceptar. Según el producto, su identidad abarca hash o ref de fuente, configuración, versión del esquema, estado de la DB, manifiesto de archivos, assets y entorno de ejecución. Una revisión favorable de un archivo con nombre `final` no puede transferirse automáticamente a otro archivo con el mismo nombre y contenido cambiado.

El flujo razonable es integrar el candidato, congelar su identidad relevante, ejecutar las pruebas físicas y la revisión independiente, y recién después presentarlo al Planner. Si se modifica materialmente después de la revisión, hay que renovar los controles afectados. No es necesario repetir toda evidencia estable cuyos inputs no cambiaron, pero sí impedir que una revisión antigua parezca avalar bytes nuevos.

El protocolo de freeze y terminal describe esa relación. El controlador de referencia puede representar transiciones de freeze, review y terminal, pero no puede fingir que un tercero aceptó un diseño ni certificar una aplicación externa que no ejecutó físicamente.

El gate técnico puede quedar aprobado y continuar pendientes una prueba con usuarios, una revisión de datos personales, un permiso de publicación, una licencia de un recurso ajeno o una aceptación final del CLIENT. Registrá cada una por separado. Un terminal estratégico significa «hay evidencia para decidir lo próximo», no «todos los efectos posteriores están autorizados».

Es útil que cada informe cite el commit, manifest y environment que realmente observó. Si la rama avanzó después, el nombre de la rama no prueba por sí solo currentness. El salto entre código, base de datos, artefactos y revisión debe ser rastreable de manera mecánica o con referencias precisas.

4. Evidencia insuficiente sin campañas artificiales

Si falta evidencia de algo ya incluido en el charter, el Planner devuelve a I3 una lista **consolidada de falsifiers**: qué se esperaba, qué ocurrió, qué candidato y evidencia lo demuestra y qué gate está afectado. I3 investiga, arregla, integra, vuelve a probar y obtiene la revisión necesaria en la **misma campaña**.

No corresponde abrir una nueva campaña porque falló un test unitario, porque el backend devuelve un error previsto, porque el reviewer detectó una regresión o porque una herramienta se desconectó. Tampoco corresponde crear PASS4, PASS5 y PASS6 para que el Planner vaya minando bugs que el Controller tiene autoridad y responsabilidad de resolver.

Cuando la misma clase de defecto reaparece en dos o más terminales, I3 debe dejar de aplicar arreglos estrechos y endurecer el subsistema de forma exhaustiva, con revisión independiente. El retorno siguiente debe demostrar que se corrigió la causa sistémica, no que se ocultó otra manifestación. Esto es disciplina de ingeniería dentro del charter autorizado.

La situación cambia cuando aparece una regla de negocio material no decidida, cambian los permisos del CLIENT o se descubre una incompatibilidad estratégica que invalida el charter. Ahí el Planner vuelve a evaluar el objetivo, los límites y las dependencias. El criterio no es cuán difícil se volvió el bug, sino si el trabajo sigue autorizado y responde al mismo objetivo.

Un terminal legible se puede expresar así: candidato y entorno exactos; recorrido real

realizado; gates aprobados con enlaces; gates abiertos con impacto concreto; riesgos nuevos; decisión pendiente; ruta siguiente y owner. Ese resumen sirve para encontrar evidencia canónica, no para instalar un segundo Product State paralelo.

5. Aceptar, recuperar o cambiar el objetivo

Aceptar. Cuando el Planner tiene los datos apropiados, registra la capacidad aceptada y actualiza las superficies canónicas y punteros `CURRENT` que correspondan. El CLIENT ejerce su aceptación reservada cuando hay calidad perceptual, comportamiento de producto o efectos externos. Solo entonces pueden depender de esa capacidad los trabajos posteriores que la necesiten.

Recuperar después de una interrupción. Si cambia el ejecutor o termina la sesión, el nuevo actor reconstruye campaña, checkpoint, candidato, owner y evidencia desde el repositorio y estado autorizado. No marca la capacidad terminada porque venció el tiempo de servicio. Tampoco crea una versión competidora del charter para continuar más rápido. La continuidad no requiere memoria perfecta del chat, pero sí un estado durable coherente.

Cambiar prioridades. Posponer un módulo puede ser un ajuste de cronograma; cambiar la condición de elegibilidad de un socio es una nueva regla de negocio. En el segundo caso se registran la decisión, el impacto y las supersesiones. Dos charters contradictorios no deben permanecer activos sobre el mismo concern. El trabajo anterior válido se conserva cuando todavía tiene valor y puede ser referenciado.

Detener por incertidumbre. Si no hay forma de relacionar un candidato con el estado actual de la DB o las fuentes, se deja HOLD para la aceptación dependiente. La investigación segura puede continuar. HOLD no equivale a «todo el proyecto está roto» ni autoriza a inventar una respuesta para destrabar la conversación.

6. Revisiones periódicas y regla de detención

Una revisión semanal debería permitirle al CLIENT preguntar: ¿qué capacidades están aceptadas?, ¿qué puedo demostrar hoy?, ¿qué falta para llegar a la próxima entrega?, ¿cuál es el riesgo de plazo?, ¿qué decisión real necesita mi intervención?, ¿el gasto y el alcance siguen justificados? El informe debe apoyarse en evidencia observable.

La actividad técnica es un medio: commits, agentes, documentos, tests, minutos de runtime y tokens no son el valor entregado. A veces un buen resultado requiere pocos cambios y pruebas bien elegidas; otras veces necesita mucha recuperación interna. Contar operaciones no permite comparar esas situaciones ni aceptar por cantidad.

El framework tiene también una regla de detención. Una vez logrados los gates de release, no agregues maquinaria genérica, políticas duplicadas o más jerarquía solo porque se puede. Un mecanismo nuevo necesita abordar una clase de falla identificada o desbloquear una capacidad. La misma regla opera dentro de la aplicación: perseguir indefinidamente microdefectos sin relevancia estratégica consume tiempo y puede deteriorar el resultado.

El equilibrio es concreto: no aceptar prematuramente una app rota, pero tampoco mantener una capacidad en limbo porque no existe un test imposible que nadie había requerido. El Planner y el CLIENT deben ver qué riesgos son materiales y qué evidencias les permiten decidir con honestidad.

7. Registro de cierre reconstruible

Una persona que no participó en la implementación tiene que encontrar lo necesario sin leer todo el historial de conversación. La ficha de cierre apunta a las superficies existentes y debería contener:

Campo	Evidencia rastreable
Candidato	Código, configuración, artifact y state exactos revisados.
Resultado	Entrada, acción y salida observada en recorrido end-to-end.
Pruebas	Resultados, comandos relevantes y entorno donde corrieron.
Datos persistentes	Readback, reinicio y recuperación cuando son aplicables.
Seguridad y derechos	Controles proporcionados a datos sensibles, licencias y exposición.
Revisión independiente	Identidad del reviewer, alcance y candidato efectivamente inspeccionado.
Cambios posteriores	Qué cambió después del freeze y qué gates se renovaron.
Aceptación humana	Decisión auténtica, no inferida a partir de un screenshot.
Ruta siguiente	Accept, retorno a I3, pregunta al CLIENT o HOLD con causa.

No copies cada campo a otro archivo `CURRENT` creado para una presentación. El registro funciona como índice de evidencia, con owners únicos y referencias que realmente resuelven. Si el enlace apunta a un artifact inexistente, a una versión anterior o a una fixture que nunca ejercitó el producto, el cierre no está debidamente sustentado.

Las familias de pruebas del propio starter también deben mantenerse separadas: corrección interna de código, contratos y transacciones por un lado; ciclo completo de instalación y recuperación en un destino portátil real por otro. Ninguna de ellas certifica por defecto la experiencia de usuario ni la lógica comercial de otra aplicación construida con el framework.

8. Ejemplo de rechazo y corrección

Supongamos que una aplicación de club ya permite registrar y desactivar socios. El reviewer encuentra que al desactivar un socio se borran sus pagos históricos, pese a que el charter exigía preservarlos. Eso es un defecto de implementación dentro del alcance ya autorizado. El Planner devuelve un falsifier concreto: «Desactivar un socio con pagos previos debe conservar transacciones después de un reinicio; en el candidato X desaparecen».

I3 inspecciona relaciones y operaciones de persistencia, corrige la causa, agrega una regresión significativa, prueba el registro después de reiniciar, integra un nuevo candidato y solicita revisión independiente. No abre una campaña «arreglar botón de desactivar». Tampoco le exige al CLIENT decidir cómo implementar una foreign key; esa es una responsabilidad técnica.

Si el reviewer revisó A y el cambio de persistencia generó B, el PASS de A no acepta B. Podrán aprovecharse pruebas que dependían de componentes intactos, pero debe renovarse la evidencia afectada por el cambio y la revisión sobre el nuevo candidato. El objetivo es evitar tanto repetición inútil como reutilización engañosa.

Ahora imaginá que el CLIENT pregunta por una función de reactivación con permisos especiales. No es la reparación del borrado de pagos: introduce comportamiento nuevo. El Planner confirma quién puede reactivar, qué pasa con el identificador anterior, cuáles son las reglas sobre pagos y si el charter actual permite esa ampliación. Una regla nueva y un fallo de una regla existente son hechos distintos.

9. Producto utilizable frente a framework verde

Una campaña puede superar validaciones de gobierno y poner el controlador de referencia en estado terminal sin que exista una pantalla usable. Para aceptar la aplicación del club, hay que abrirla, registrar un socio ficticio, encontrarlo, modificarlo, desactivarlo y verificar que su historial permitido permanece. Si la persistencia importa, reinicié el programa y comprobá el readback real. Si hay permisos, también intentá la ruta que debe ser denegada.

El release gate de ArquitecturaRepo es diferente: `tools/verify_v1_release.py` comprueba una versión concreta del framework reutilizable mediante ejecución y fixtures sintéticas. Verificación de fuentes, instalación portátil Linux/Windows y lectura de artefactos de release son controles distintos. El hecho de que el framework haya pasado no demuestra que cada proyecto que lo adopte opere correctamente ni que una integración nativa de un host tercero esté probada.

La matriz pública de soporte v1.4 distingue exactamente lo ejercitado de lo que queda fuera de la prueba portátil. Evitá frases como «todo pasó» sin especificar candidato, escenario, entorno y gate. Un PASS limitado puede ser un progreso real; exagerarlo lo convierte en una fuente de error de producto.

10. Qué hacer después del cierre

Tras la aceptación estratégica, persistí los cambios en Product State y referencias actuales, y desbloqueá únicamente los downstream que dependen de lo demostrado. Ante un rechazo in-charter, devolvé a I3 la reparación consolidada. Ante una invalidación material, revisá el límite con el owner estratégico. Ante currentness no verificable, declarás HOLD concreto y seguís con las verificaciones seguras posibles.

El CLIENT debería poder terminar el día sabiendo qué existe realmente, qué no está listo, quién lo está resolviendo y cuándo se revisará el siguiente objetivo. Si esa información solo se puede reconstruir abriendo el chat de ayer, la arquitectura aún no cumplió su promesa de continuidad.

En cada cierre conviene preguntarse si la evidencia sería convincente para una persona escéptica que no participó del desarrollo. No hace falta escribir un tratado por feature: hacen falta IDs correctos, pruebas apropiadas, resultados observados y una ruta siguiente clara. Esa austeridad documental protege la velocidad y la confianza al mismo tiempo.

Siguiente: capítulo 07, ejemplo completo de una aplicación para un club.

Fuentes y vigencia

- Integridad de revisión, freeze y terminal
- Ciclo de capacidades y evidencia
- Validación ejecutable
- Gates de release del framework
- Soporte y distribución v1.4
- Verificador de release

La aceptación siempre depende del candidato y producto reales. Los ejemplos no reemplazan pruebas ni conceden autoridad a quien no la tiene.

07 — Ejemplo guiado: una aplicación de gestión para un club

> Este capítulo es un **ejemplo didáctico ficticio**, no una aplicación ya implementada ni un backend generado. Los datos, decisiones, nombres de campañas y resultados esperados son ilustrativos. En un proyecto real, requisitos, repositorio, DB, runtime, permisos y evidencia de aceptación tienen que provenir de sus fuentes vigentes.

Contenido del capítulo

1. El producto que quiere construir el club
2. Descubrir preguntas que cambian el resultado
3. Instalar el control sin confundirlo con la app
4. Primera capacidad: gestión de socios
5. Del Planner a I3: una solicitud operable
6. Qué probar antes de aceptar
7. Segunda capacidad: cuotas y pagos
8. Evaluar el ejemplo completo
9. Primer ciclo con datos ficticios
10. Registrar y enrutar un defecto
11. Dependencias del segundo ciclo
12. Ejercicio final de recuperación

1. El producto que quiere construir el club

Imaginá un club de barrio con una persona administrativa que registra socios, cobra cuotas, administra actividades y anota participantes. Hoy usa planillas dispersas y anotaciones. Quiere una aplicación sencilla que conserve el historial, permita encontrar a cada socio, evite superar la capacidad de una actividad e informe quién tiene pagos al día.

La primera versión funcionará para operadores internos. Para controlar alcance, el CLIENT decide no incorporar todavía pagos en línea, una aplicación móvil separada, facturación fiscal, notificaciones masivas ni integraciones externas. No es que esas funciones sean malas; simplemente no resuelven el primer problema que se quiere validar y aumentarían tiempo, seguridad y coste.

Esa descripción propone una dirección, pero no decide el lenguaje, la base de datos, el hosting ni los permisos exactos. Tampoco significa que el framework cree una interfaz de socios al instalarse. Esas decisiones dependerán del repositorio real, del presupuesto, del ambiente y del criterio de ingeniería una vez que el CLIENT confirme los objetivos de producto.

El flujo probable de capacidades es identidad de socios, luego cuotas e historial, después actividades/cupos y finalmente inscripciones basadas en elegibilidad. No son cuatro aplicaciones ni obligatoriamente cuatro equipos. Son dependencias: no

conviene asignar pagos a socios sin identidad consistente, ni habilitar inscripciones por deuda cuando todavía no existe un estado de cuotas confiable.

La arquitectura tiene valor si permite conservar esas decisiones y sus consecuencias durante varias semanas de trabajo, con agentes diferentes y sesiones interrumpidas. Su objetivo no es dibujar una organización compleja, sino mantener claras las interfaces y la autoridad para construir algo realmente útil.

2. Descubrir preguntas que cambian el resultado

El Planner entrevista al CLIENT: ¿quién usará la aplicación?, ¿funciona en una sola computadora o en la red?, ¿cuántos socios hay?, ¿qué información personal se almacena?, ¿qué respaldo necesitan?, ¿hay personal con distintos roles?, ¿cuánto tiempo y dinero hay disponibles? Cada respuesta cambia decisiones de riesgo, interfaces o tecnología.

Una regla inicial propuesta podría ser: «Un administrador puede desactivar un socio, pero sus pagos e inscripciones históricas siguen disponibles para el personal autorizado». Antes de implementarla se confirma qué significa «desactivar», quién está autorizado, qué aparece en una búsqueda de socios activos y cómo se conserva el historial.

Otras dudas deben hacerse visibles: ¿una cuota vencida impide inscribirse?, ¿qué pasa con un acuerdo de pago?, ¿se libera el cupo apenas una persona cancela?, ¿hay lista de espera?, ¿se puede reactivar una membresía?, ¿puede corregirse un pago ya emitido?, ¿cuándo se considera cerrado un período? Adivinar esas reglas desde la implementación produciría comportamientos difíciles de corregir después.

Área	Comportamiento propuesto para la primera versión
Socios	Crear, editar, buscar y desactivar, conservando relaciones históricas.
Cuotas	Registrar pagos, identificar períodos pendientes y consultar historial.
Actividades	Definir nombre, horario, cupos y plazas disponibles.
Inscripciones	Inscribir y cancelar con reglas explícitas de deuda y capacidad.
Fuera de alcance	Pago online, app móvil, avisos masivos y dashboards elaborados.

Esta tabla no es un requisito vigente hasta que el CLIENT la confirme. La necesidad de guardar transacciones sugiere evaluar el perfil `STATEFUL`, pero incluso ese perfil requiere una adopción adecuada: definir owners de datos, migraciones, idempotencia, seguridad, recuperación y evidencia física proporcional.

3. Instalar el control sin confundirlo con la app

En un repositorio de club nuevo y sin control canónico previo, se puede preparar una capa de gobierno. Desde un checkout de ArquitecturaRepo v1.4, por ejemplo, en PowerShell:

```
& "python" "tools\scaffold_project.py" --target "..\club-app" --profile "STATEFUL" --p
```

El comando crea `.agentic-sdlc/` dentro de `club-app/`, en estado inicial `HOLD / UNTRUSTED_CONTEXT`. No crea el backend, no inventa una DB, no demuestra que existe una pantalla, no configura automáticamente un LLM y no autoriza ejecutar una campaña de desarrollo. La capa de control es infraestructura para organizar el trabajo de producto; la aplicación sigue siendo el resultado a construir.

Si `club-app/` ya tiene un sistema de gobierno canónico, el procedimiento es distinto: se usa el preflight read-only de proyectos gobernados. No hay que instalar otro Product State encima, reemplazar un controlador existente ni duplicar la autoridad para que el ejemplo funcione.

Después del scaffold, se ejecutan los comandos de inspección, validación e inventario indicados por el instalador y el capítulo 01. Planner e I3 leen la aplicación que realmente existe, reconstruyen requisitos y baseline, preparan una especificación revisada de adopción, hacen preflight, promueven los registros autorizados y verifican que una sesión nueva pueda reconstruir todo sin depender del chat.

El CLIENT revisa alcance, experiencia esperada, presupuesto y fecha. Ingeniería propone stack, modelo de datos, red, backups, permisos y plan de entregas. La primera campaña material se autoriza solo cuando la base y el estado vigente son coherentes. Si faltan datos, se mantiene un HOLD concreto para las operaciones que dependan de ellos.

4. Primera capacidad: gestión de socios

¿Por qué comenzar por socios? Porque cuotas e inscripciones necesitan una identidad estable a la cual referirse. Una capacidad inicial bien delimitada podría decir:

> Una persona administrativa autorizada puede registrar un socio, encontrar su ficha, corregir datos, consultar el estado y desactivarlo, sin destruir pagos e inscripciones históricas que deban conservarse.

El charter de esta capacidad especifica sus interfaces, dependencias, inputs/outputs, ownership de datos, comportamiento de errores y gates aplicables. No necesita imponerle al ejecutor los nombres de todas las funciones ni una lista microscópica de commits. I3 organiza la implementación, las pruebas, refactors internos y la revisión del candidato integrado.

La lista de casos relevantes incluye identificadores duplicados, datos inválidos, escritura seguida de lectura real, recuperación tras reinicio, búsquedas con filtros activos/inactivos, preservación de relaciones, autorización de operaciones y recorrido completo de interfaz si existe una UI. Los ensayos deben registrar ambiente, candidato y resultado observable. Una prueba aislada de validación del esquema no basta para afirmar que el usuario pudo realizar el flujo completo.

El CLIENT no tiene por qué decidir el ORM, el tipo de índice o los nombres de las tablas. Ingeniería sí debe pedir confirmación cuando una decisión técnica tenga consecuencias de producto o coste: uso sin internet, exposición de datos, restauraciones, derechos de acceso, dependencia de un proveedor externo o presupuesto de operación.

Los registros existentes —si el club ya usa una base— son evidencia y datos que hay que proteger. No se toman las fichas de este manual como si fueran datos reales; tampoco se copian muestras con información personal hacia un issue público. Una migración necesaria se diseña y prueba con rollback y respaldo autorizado, no se

presupone segura porque el scaffold pasó.

5. Del Planner a I3: una solicitud operable

El mensaje al Controller debe ser íntegro y suficientemente compacto para ejecutar. Un ejemplo de estructura, no una campaña real, es:

> **Goal:** Entregar la capacidad aprobada de gestión de socios.
>
> **Context:** Reconstruir requisitos confirmados, Product State canónico, repositorio actual y ref exacto. Identificar registros afectados y estado vigente de DB.
>
> **Constraints:** Conservar historial, respetar permisos y presupuesto; no agregar cuotas, notificaciones ni funciones no autorizadas.
>
> **Execution gates:** Probar físicamente alta, búsqueda, edición, baja lógica y persistencia tras reinicio; cubrir datos inválidos, duplicados, permisos y concurrencia relevante. Revisar independientemente el candidato integrado.
>
> **Done when:** Entregar identidad exacta, evidencia física, tests aplicables, hallazgos del reviewer, gates abiertos y terminal estratégico honesto.

La campaña contiene el resultado y sus límites. I3 puede organizar tareas, investigar fallos y volver a probar sin pedir una nueva autorización por cada error. Si el doble clic crea dos socios idénticos pese a una regla de unicidad confirmada, se corrige dentro de la campaña. No existe razón para abrir una segunda campaña estratégica llamada «evitar doble clic».

Cuando una operación implica varios cambios delicados o mucho coste, el Controller puede preparar una unidad acotada de trabajo con inventario, presupuestos y condiciones de parada. Eso no cambia las decisiones de producto ni concede permiso para ejecutar un proveedor de pago. La autoridad de una unidad técnica deriva del charter y no puede superar sus límites.

6. Qué probar antes de aceptar

Tomá una ficha completamente ficticia con el identificador `SOCIO-001`. La siguiente tabla enumera acciones esperadas para una futura aplicación: **no es evidencia de que exista ni de que ya se haya ejecutado**.

Paso	Acción del usuario	Resultado que se debería observar
1	Abrir la app como administrador autorizado.	El mecanismo de acceso real permite el rol correcto.
2	Crear `SOCIO-001` con datos sintéticos válidos.	Se guarda exactamente una ficha identificable.
3	Buscar por identificador permitido.	Aparece la ficha correcta sin filtrar datos de otros roles.
4	Editar un teléfono ficticio.	La nueva información figura al guardar y releer.

Paso	Acción del usuario	Resultado que se debería observar
5	Cerrar y abrir la aplicación.	El cambio persiste mediante la DB y el camino de recuperación reales.
6	Desactivar al socio.	Ya no figura como activo; su identidad histórica permanece.
7	Consultar pagos anteriores con permisos.	Se conserva historial y se expone solamente al rol previsto.
8	Intentar una operación inválida o duplicada.	Se rechaza de forma clara sin corromper datos.

Para cada paso anota entrada, acción, esperado, observado, candidato, fecha y entorno. La ausencia de excepciones en una consola no demuestra persistencia. Es necesario releer el estado de forma independiente y, si importa la durabilidad, reiniciar el proceso que guarda los registros.

Si varios empleados pueden operar a la vez, se ensayan acciones solapadas o concurrencia adecuada a la arquitectura. Un test secuencial no prueba seguridad ante dos operaciones simultáneas. Si hay datos personales, se utilizan registros ficticios en capturas y demos; los secretos de acceso nunca se incorporan al repositorio.

La evaluación perceptual debe hacerla una persona competente cuando la experiencia de usuario tenga importancia. Esa prueba no reemplaza los tests de backend ni la inspección de DB. Del mismo modo, una prueba de DB no prueba que alguien encuentre un botón o comprenda el mensaje de error. Cada gate tiene su propósito y su resultado.

7. Segunda capacidad: cuotas y pagos

Una vez aceptada la identidad de socios, el club solicita cuotas. Ahora el Planner confirma períodos de pago, vencimientos, deudas parciales, correcciones, permisos, idempotencia y quién puede revisar o revertir una operación. El estado «al día» debe surgir de reglas explícitas y datos actuales, no de un cálculo improvisado en cada pantalla.

Una decisión posible del CLIENT es: «Un pago cargado por error se revierte con un asiento de auditoría; el registro original nunca se borra». Tiene consecuencias de producto, privacidad y arquitectura. El Planner persiste la decisión; I3 implementa las operaciones, esquema, restricciones, pruebas, recuperación y evidencia necesaria.

La repetición de una solicitud —por red inestable, doble clic o reintento automático— no debe generar un cobro duplicado si la política acordada exige una operación idempotente. Eso se prueba en los límites adecuados, no solamente llamando una función en aislamiento. Si la aplicación procesa dinero real, los gates de autorización, auditoría, seguridad y publicación necesitan un tratamiento proporcional y explícito.

Las actividades e inscripciones se construyen después de tener identidad y estado de cuotas confiables. No se agrega por moda un conjunto de microservicios, un procesador de pagos cloud, una aplicación móvil o un sistema complejo de roles.

Cada elemento adicional debe justificar el riesgo, coste y beneficio. La meta de la primera versión es que alguien del club resuelva tareas concretas.

8. Evaluar el ejemplo completo

Al terminar, deberías poder distinguir qué decidió el CLIENT, qué requisitos confirmó el Planner, cómo I3 desarrolló dentro del charter, qué evidencia física y de DB se produjo, qué revisó una persona independiente y quién puede aceptar cada gate. También deberías entender por qué los tests verdes del framework no hacen automáticamente aceptable la aplicación construida con él.

El objetivo no es memorizar nombres internos de JSON o enumerar todas las políticas. Es conocer dónde vive la verdad vigente, quién puede cambiarla, cómo se demuestra un comportamiento y qué hacer cuando el contexto se pierde. Una arquitectura con roles elaborados pero sin recorrido de socio demostrable no entregó la capacidad solicitada.

La misma idea se aplica a otros productos: una API CRUD, un servicio de reservas, un dashboard de un negocio, una aplicación educativa o un juego. Lo que se debe demostrar cambia según el producto, pero la separación entre intención, autorización, implementación, evidencia y aceptación sigue siendo útil.

9. Primer ciclo con datos ficticios

Para practicar, inventá a «Persona de prueba A» y un número de contacto obviamente ficticio. Preguntá qué datos son obligatorios, qué identificador debe ser único y quién puede modificarlos. El Planner transforma decisiones confirmadas en requisitos verificables. Ingeniería selecciona persistencia, controles de unicidad, manejo de concurrencia y transacciones.

Recorré todo el circuito propuesto: alta, listado, búsqueda, edición, reinicio, readback, desactivación y consulta autorizada del historial. Anotá qué ocurrió realmente en cada etapa. Una captura al crear el socio es el comienzo del ensayo, no una prueba de que la DB guardó el cambio. Si todavía no programaste la app, marca esos pasos como `NOT RUN`, nunca como PASS.

Agregá pruebas negativas: un identificador repetido, campo inválido, desconexión controlada de base durante el guardado y dos envíos del mismo formulario. Las respuestas esperadas dependen de las reglas confirmadas, pero no debería generarse un estado silenciosamente contradictorio. En un producto con varios operadores, verificá condiciones de carrera que afecten registros o cupos.

El feedback de la primera prueba puede modificar detalles de experiencia: orden de campos, claridad de una advertencia, lugar de un botón. Si una observación implica una regla de negocio nueva, corresponde al CLIENT y Planner; si revela un defecto bajo reglas existentes, I3 lo repara en el charter. La frontera se decide por el significado de la observación, no por el cargo de quien la encontró.

10. Registrar y enrutar un defecto

Supongamos que el CLIENT desactiva a «Persona de prueba A» y descubre que el historial de pagos desaparece. Si conservarlo era requisito aprobado, esto es un

defecto in-charter. La evidencia debe indicar candidato exacto, pasos de reproducción, datos ficticios usados, resultado observado y gate afectado.

El Planner consolida el falsifier y lo entrega a I3. El Controller investiga las relaciones de persistencia, implementa la corrección causal, agrega regresión, prueba readback después de reinicio, integra la versión nueva y solicita la revisión apropiada. No crea campaña paralela, no pide al CLIENT determinar cómo funciona una clave foránea y no acepta un PASS emitido para los bytes anteriores.

En otro momento el CLIENT pregunta si un socio inactivo puede recuperar el mismo número al ser reactivado. Si no había una regla confirmada, no se puede llamar bug a una de las respuestas posibles. El Planner hace la pregunta de producto, registra la nueva decisión, examina impactos y determina si puede resolverse dentro de los límites actuales o necesita revisar la estrategia.

El manual no sustituye hechos. Ninguna de estas historias prueba que en algún repositorio real haya sido implementada una operación. Lo correcto es usar el ejemplo para aprender qué testigos, contratos y autorizaciones pedir antes de aceptar un resultado equivalente.

11. Dependencias del segundo ciclo

Las cuotas necesitan identificador estable de socio y reglas claras de pagos parciales, duplicados, vencimientos y correcciones. El Planner registra su definición usando la capacidad de socios aceptada; I3 trabaja con el candidato e interfaces actuales, no con una implementación vieja recordada de una conversación.

Antes de implementar la inscripción a actividades, hay que comprobar que el sistema realmente produce un estado de cuotas consistente y consumible. Si la elegibilidad depende de una deuda mal definida, una inscripción puede aceptarse en una pantalla y rechazarse en otra. El objetivo del diseño por dependencias es evitar dos lógicas de negocio contradictorias.

El orden de capacidades también ayuda a conversar con el CLIENT. Un hito visible de socios puede probarse antes de invertir en pagos; un hito de cuotas puede validarse antes de un calendario de actividades. Así se obtiene feedback temprano, se detecta un problema en el modelo cuando todavía es barato corregirlo y se protege la fecha de entrega.

No confundas una dependencia de producto con la necesidad de multiplicar agentes. Un único Controller competente puede orquestar implementación y QA; el Planner conserva estrategia y aceptación. Los especialistas o revisores independientes se usan cuando reducen un riesgo material, con scope, evidencia y autoridad claramente limitados.

12. Ejercicio final de recuperación

Imaginá que el trabajo se interrumpe hoy y mañana entra un Planner nuevo que no leyó este chat. El repositorio y el runtime autorizado deben permitir reconstruir qué capacidad de socios se aceptó, qué candidato y evidencias respaldan esa aceptación, qué preguntas sobre cuotas siguen abiertas, quién posee la ejecución y cuál es la próxima acción permitida.

Si dos campañas reclaman el mismo recurso, hay que conciliar autoridad antes de escribir. Si el último informe dice `CURRENT` pero apunta a una versión anterior, la discrepancia se investiga; no se decide por el mensaje más reciente. Si falta un testigo

de DB o una fuente comprometida, se establece HOLD específico para el trabajo dependiente y se siguen realizando comprobaciones read-only permitidas.

Para cerrar el ejercicio, clasificá cada evento como decisión nueva de producto, defecto ordinario dentro del charter, fallo de entorno, permiso externo pendiente o invalidación estratégica. Esa clasificación define owner y ruta. Si podés reconstruir el siguiente paso sin depender de la memoria de una conversación, el framework comenzó a cumplir su propósito.

Lo más importante del ejemplo no es haber instalado `.agentic-sdlc/`. Es transformar una intención humana en capacidades demostrables, sin perder coherencia después de cambios, interrupciones y revisiones. La prueba definitiva está en el producto que alguien usa y en la evidencia que otra persona puede comprobar.

Fuentes y vigencia

- Ejemplo sintético mínimo del repositorio
- Fork y bootstrap
- Discovery, requisitos, stack y delivery
- Límite de decisiones del CLIENT
- Estándar de Engineering Campaign Charter
- Inicio público y soporte v1.4
- Preflight de repositorio ya gobernado

Los pasos y resultados de la aplicación del club son didácticos. No representan un sistema desplegado, una capacidad aceptada ni una integración de IA o servicio externo.